

Region Convolution Neural Network

R-CNN Fast R-CNN Faster R-CNN

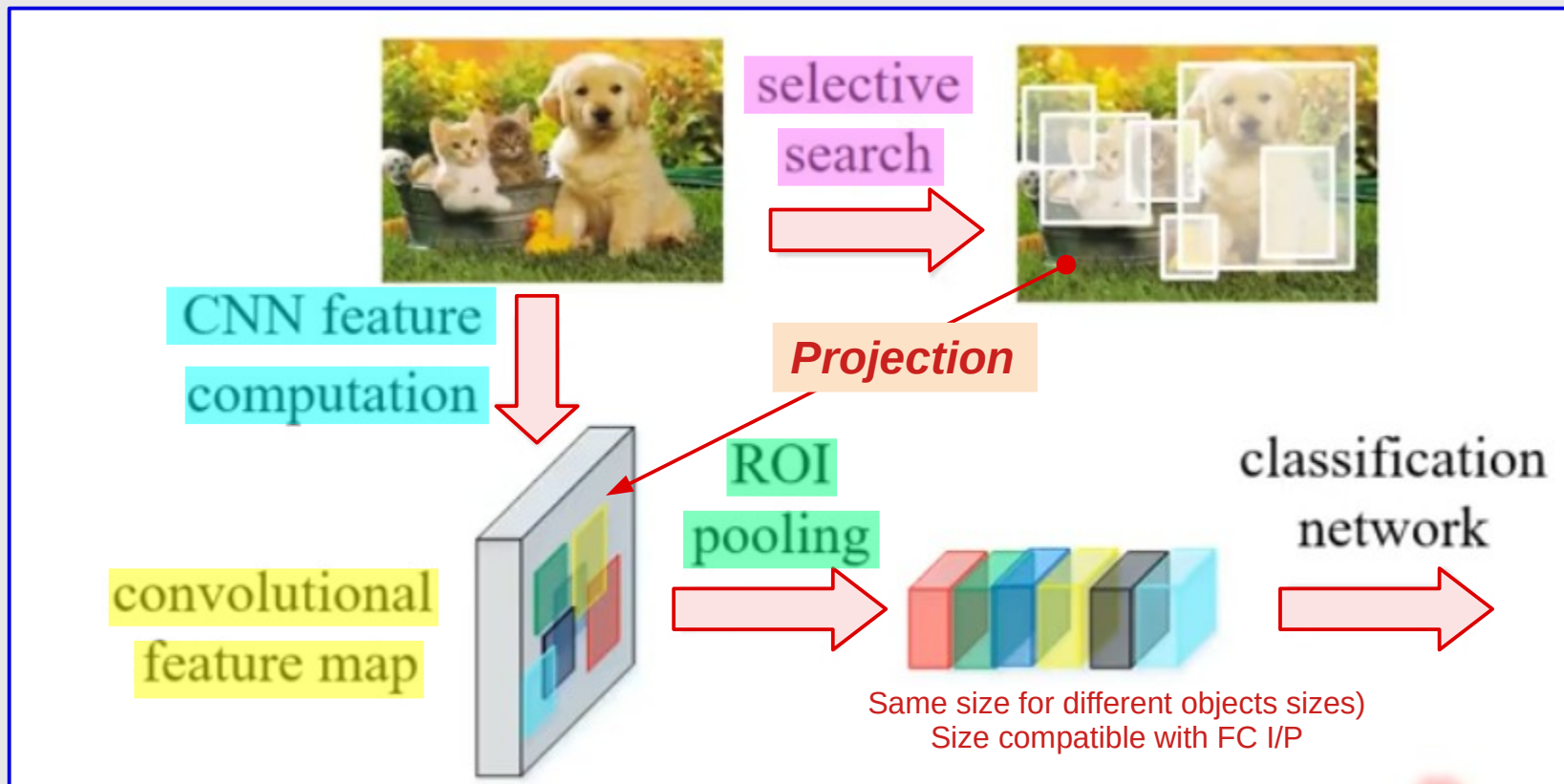
Dr. Khaled Mostafa ElSayed
kmelsayed@zewailcity.edu.eg

Region Proposal Network (RPN)

Review of Fast RCNN

Fast RCNN

(*Extract Regions from Feature Map Domain*)



Disadvantages of Fast RCNN

- **Slow** (due to using **selective search** technique for region proposal)
- Still **large number of Proposed regions**
- Same object will be detected in multiple windows (with different Bounding Boxes)

Solution

- **Automate Region Proposal using Trained Network** (instead of handcrafted techniques such as Selective Search and Edge Boxes)
- **Optimize** number of Proposed Regions

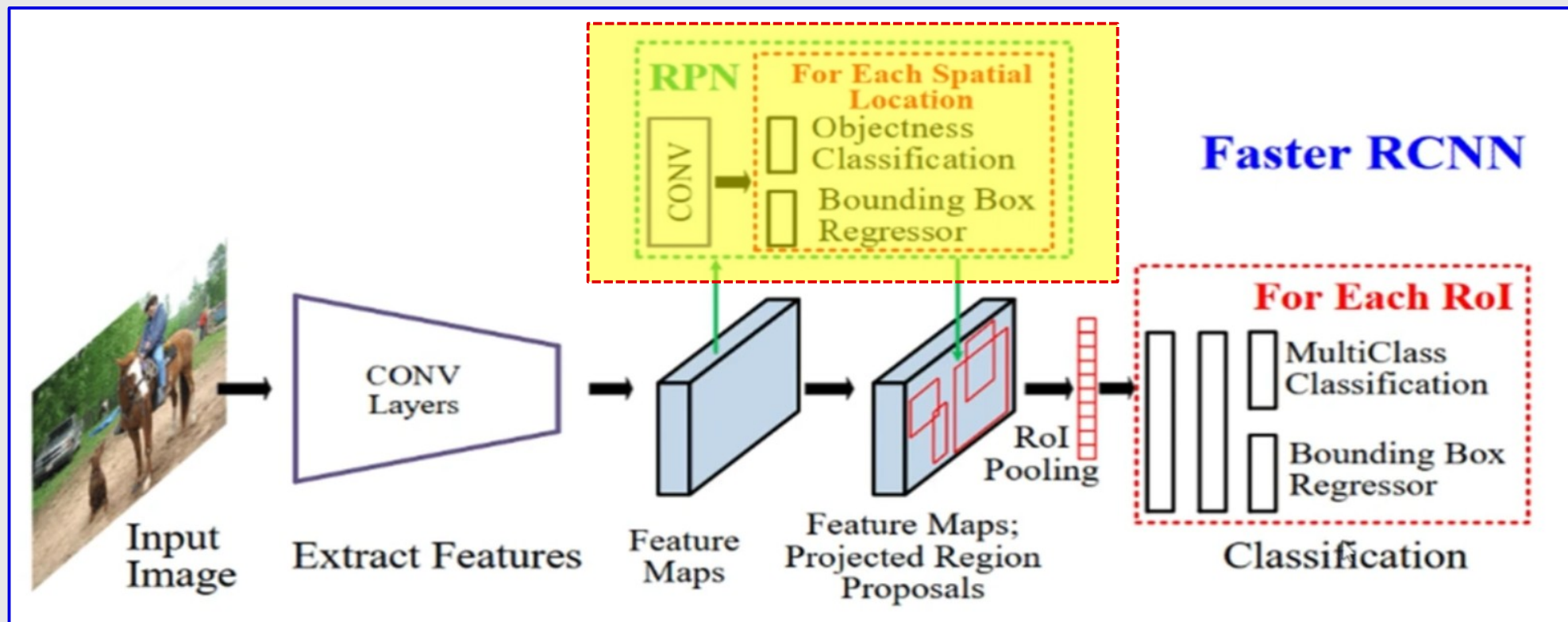
Faster RCNN

(Region proposal Network (RPN) + CNN)

- Region Proposals through *the trained “Region Proposal Network RPN”*
The whole image is passed through CNN to get feature map of the whole image
- Each region is extracted from feature map (not from the original image). In other words, Project the bounding box on the Feature map of the image.
- Since regions are of **different sizes**, their corresponding feature map will be of different sizes. Can't be feed to FC NN directly (required to be of same size)
- Instead of cropping regions in RCNN; Apply **ROI pool** layer to fix length of Feature maps of all different-size regions.
- fixed length Feature Maps are passed to FC NN for classification and regression of Bounding Boxes.

Region Based CNN

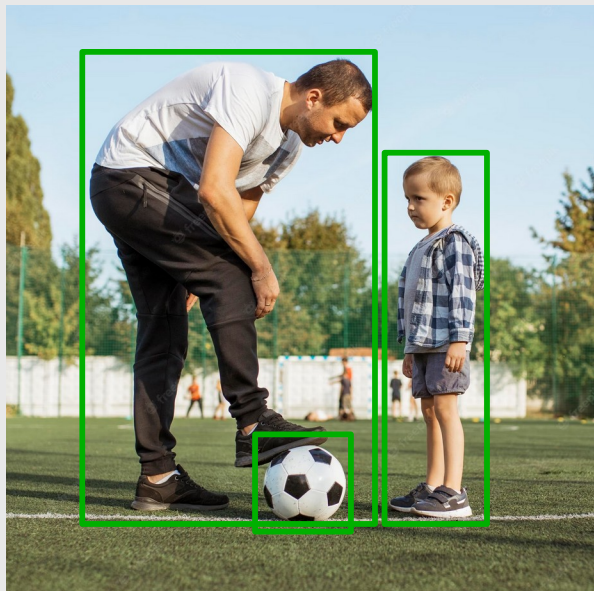
FASTER R-CNN (Region proposal + CNN)



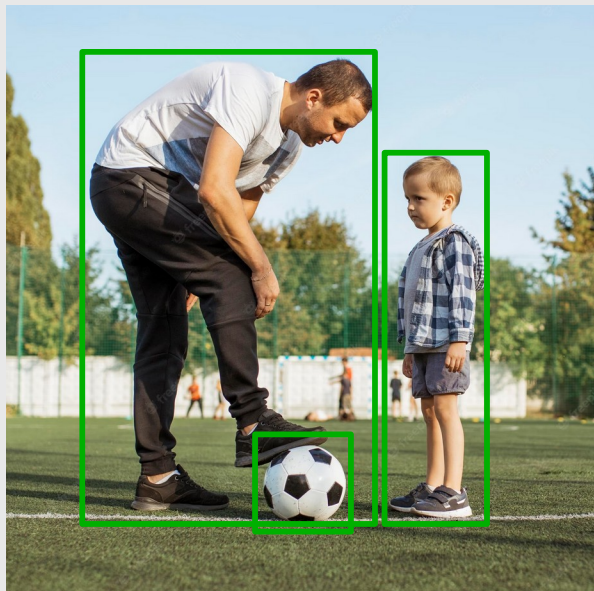
Region Proposal Network

RPN

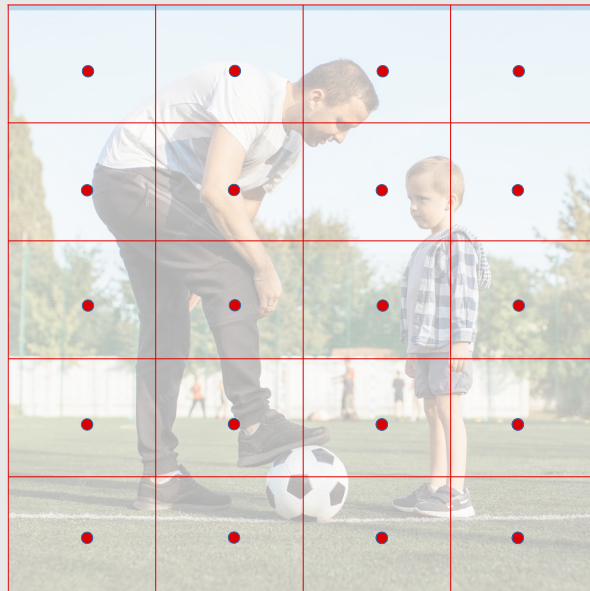
[1] Concept of Anchor Points and Anchor Boxes



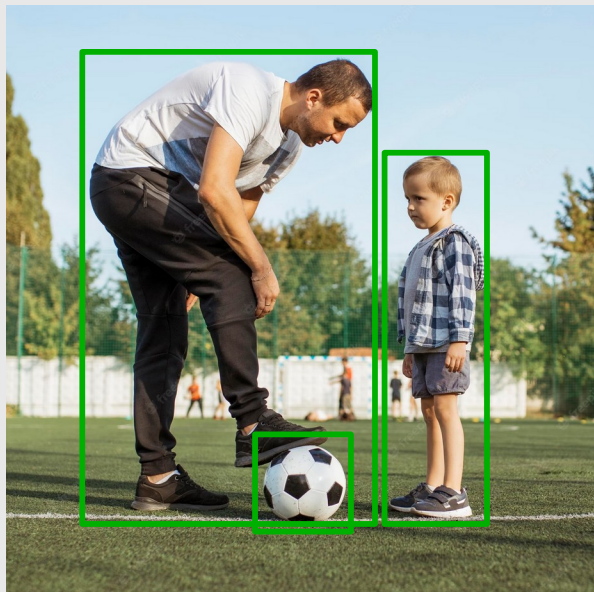
Input Image



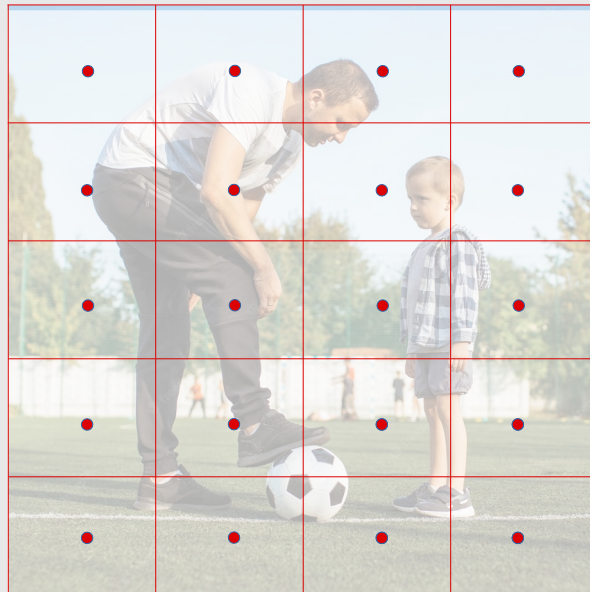
Input Image



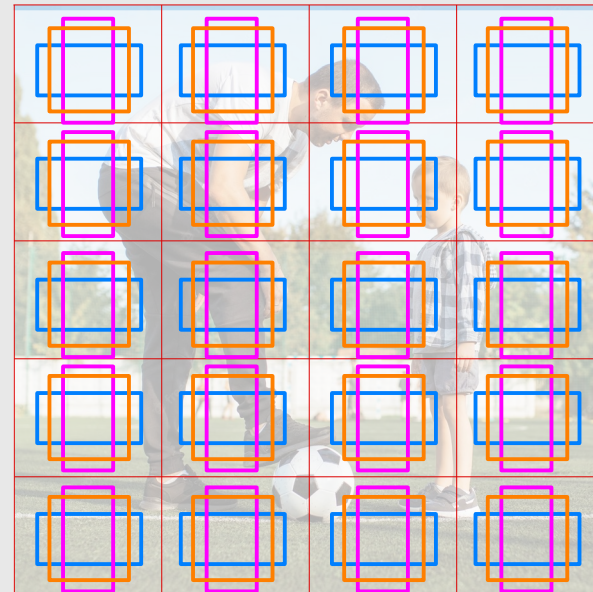
Define Anchor Points



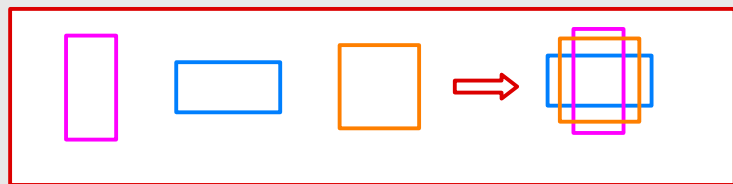
Input Image



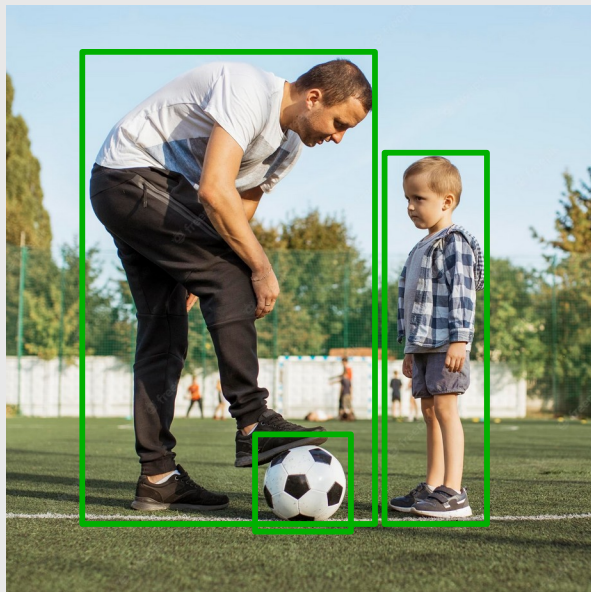
Define Anchor Points



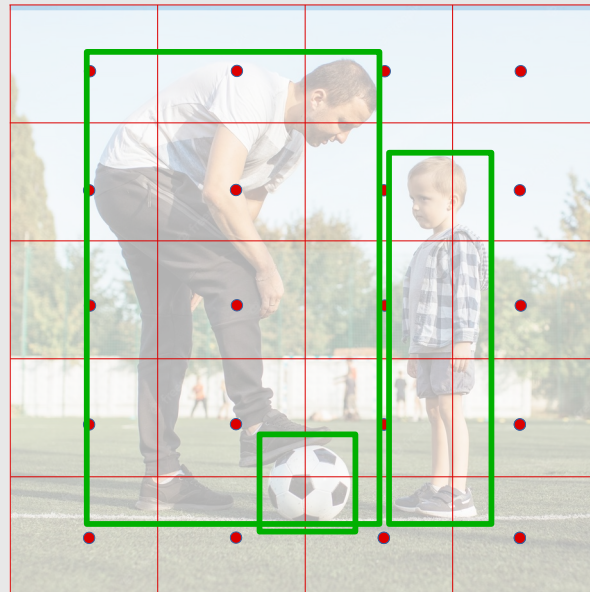
*Define Anchor Boxes
(centered at Anchor Points)*



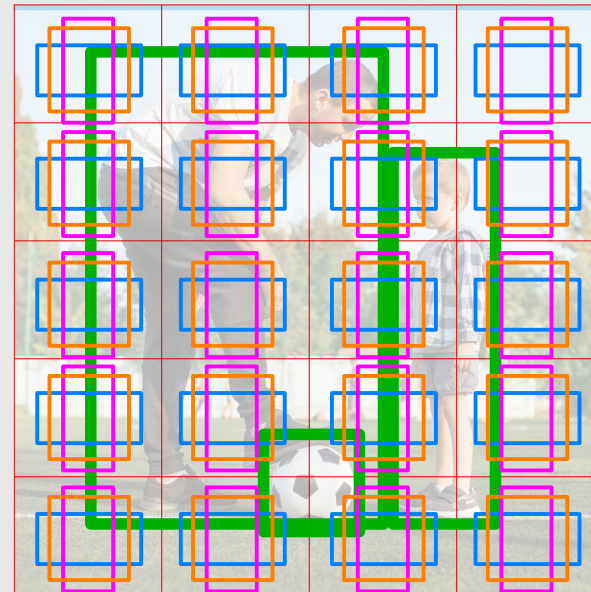
*Define Anchor Boxes
Different Scales, Different Aspect ratios
In original paper: 3 scales, 3 ratios
(9 anchor boxes)*



Input Image

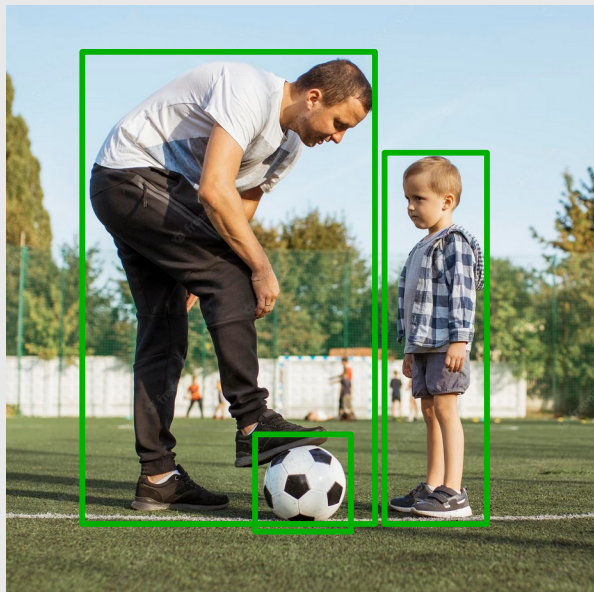


*Define Anchor Points
(20 Points)*

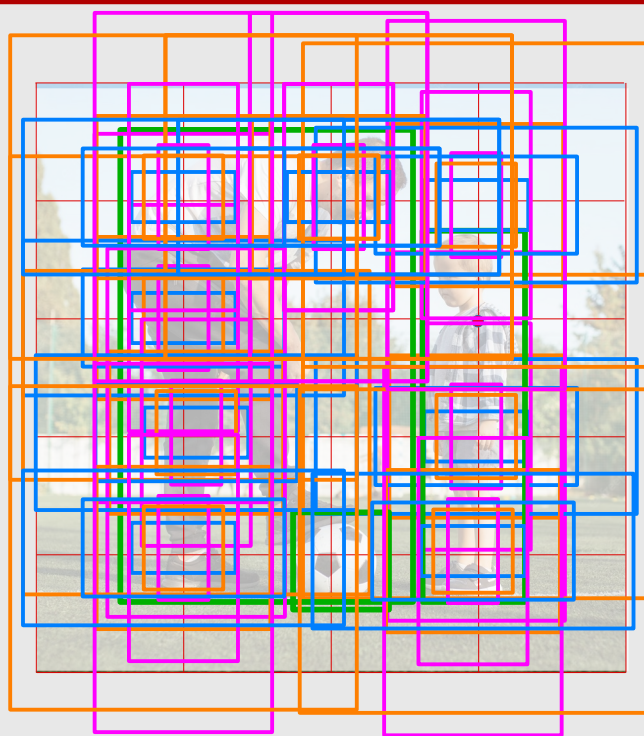


*Define Anchor Boxes
(centered at Anchor Points)*

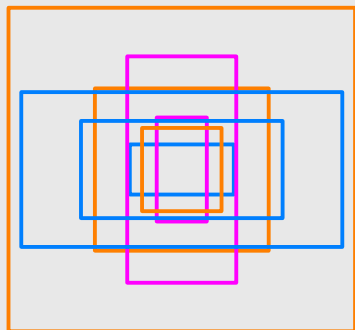
- *Selected sizes of Anchor boxes are too small (with respect to objects)*
- *Only **Ball** may be detected by anchor boxes*
- *Other anchor boxes will be assumed as background (IOU with ground truth is small)*



*Input Image
with Ground Truth*

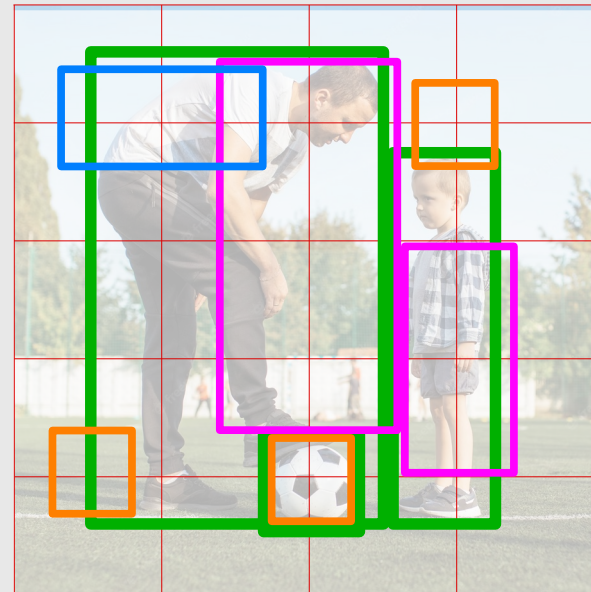


Define Anchor Points



*Define Larger Anchor Boxes
3 scales, 3 ratios (9 anchor boxes)*

Note: Many Overlaps between boxes (No Problem)



*Example of Some Anchor boxes with
High IOU (**Objects**)*

*Others with small IOU (**Background**)*

Anchor Boxes classification

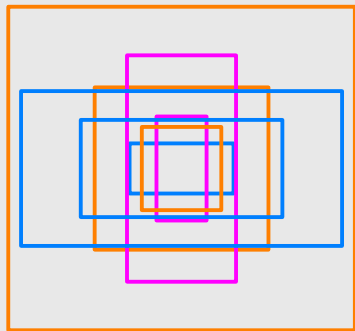
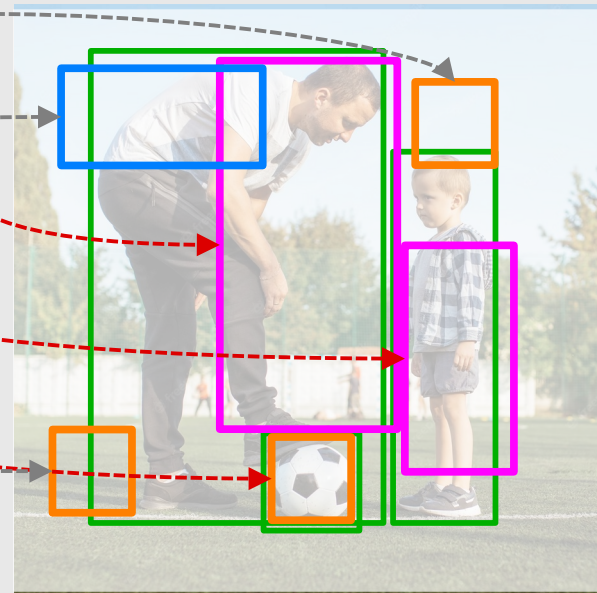
15

*Some Anchor boxes with High IOU
(Objects)*

Object

*Some Anchor Boxes with small IOU
(Background)*

Background



*Define Larger Anchor Boxes
3 scales, 3 ratios (9 anchor boxes)*

*Note: Many Overlaps between boxes
(No Problem)*

Number of Anchor Boxes (in the above example):

If $K=9$ Anchor boxes per anchor point (3 scales * 3 ratios)

of anchor points = $4 * 5 = 20$

Total Number of Anchor boxes (within Image)= $20 * 9=180$ boxes

Use Different Scales

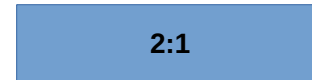
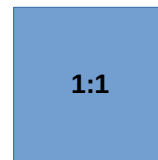
For Example: 128 x 128 256 x 256 512 x 512

Three ratios for each scale

1:1

2:1

1:2



[2] Anchor Boxes Parameters

Anchor Boxes Parameters

Each Anchor Box is characterized by two vectors:

[1] Content Classification vector:

- Enclose Object
- Part of the Background
- Mix between part of object and part of background

Classification (or ***Objectness***) Vector Length=2

[2] Bounding Box offsets vector [w.r.t Ground truth box of its enclosed object]

(for Anchor boxes classified as containing objects ONLY)

- Displacement between both centers
- Ratio between Width and height of Anchor Box and Ground Truth Box

Regressor Vector Length = 4

[1] Classification of Anchor Boxes

Assign class label to each anchor as:

- Anchor Box encloses Object (**Positive** Anchor Box)
- Anchor Box encloses Background (**Negative** Anchor Box)
- Anchor Box encloses part of Object and Part of Background (**Mix** Anchor Box)

Remember:

*Classification of Anchor Boxes is based on IOU Between
Anchor Box and Ground Truth Object Boxes in the image*

[1] Classification of Anchor Boxes

Classification of Anchor Boxes is based on IOU Between Anchor Box and Ground Truth Object Boxes in the image

Anchor Box is classified as Positive if:

- 1- Has the **Highest IOU** with one of the ground truth boxes (one of the objects)
- 2- Has **$\text{IOU} \geq 0.7$** with any of the ground truth boxes (even if not the Highest)

Anchor Box is classified as Negative if:

- 1- Has **$\text{IOU} \leq 0.3$** with ALL ground truth boxes (all objects)

Anchor Box is classified as Mix (Undetermined) if:

- 1- Neither Positive Nor Negative

[1] Classification of Anchor Boxes

Positive



Highest IOU with the Father Object

$$\begin{bmatrix} 1 \\ 0 \end{bmatrix}$$



IOU > 0.7 with the Father Object (but not the highest)

Negative



IOU < 0.3 with ALL Objects

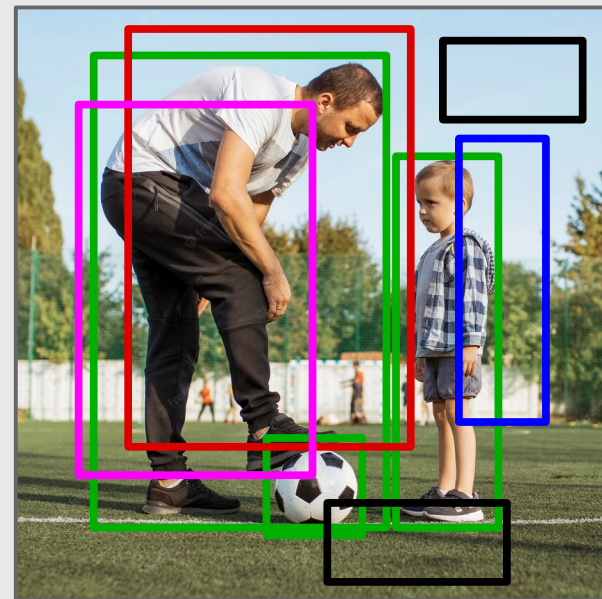
$$\begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

Undetermined (Mixed)



0.3 < IOU < 0.7 with ALL Objects

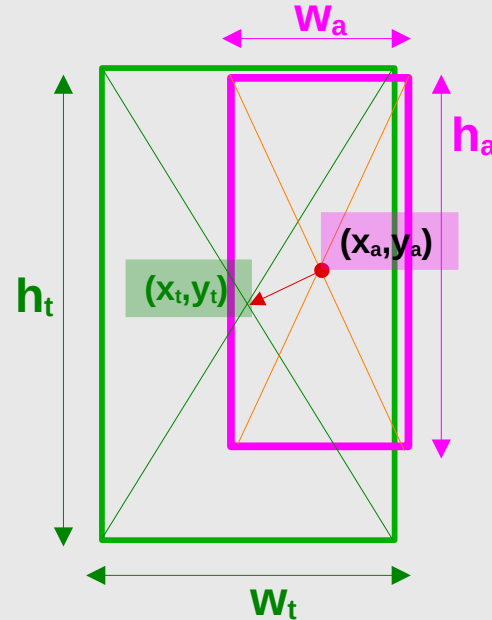
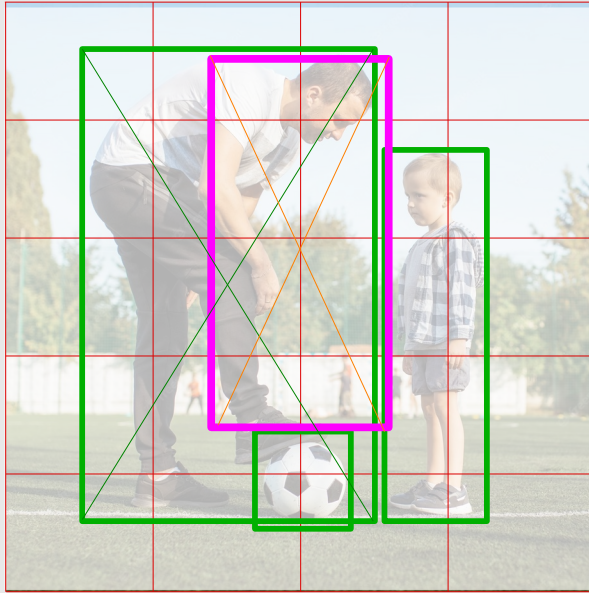
$$\begin{bmatrix} 0 \\ 0 \end{bmatrix}$$



Object (Ground Truth)

[2] Regressor Vector of Anchor Box

21



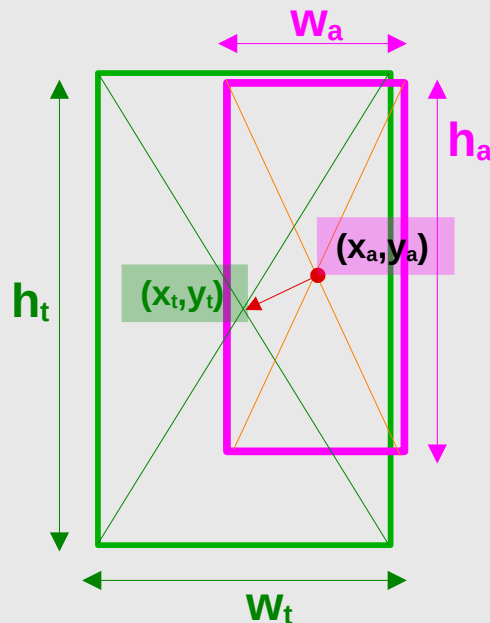
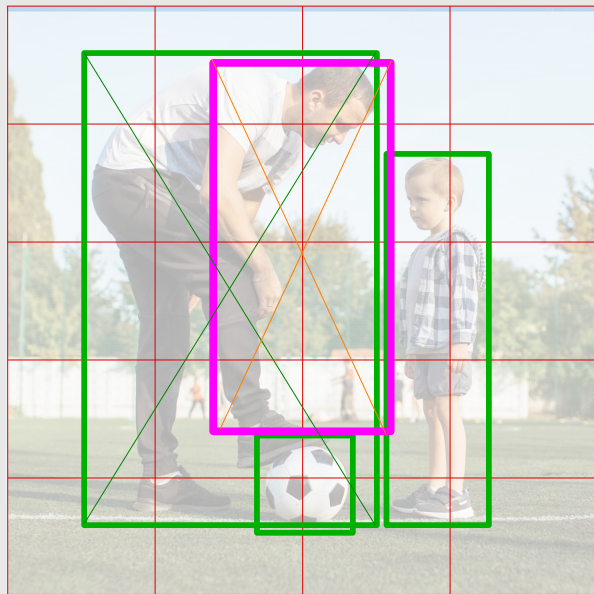
***Regressor Vector**
contains $[X, Y, W, H]$
of each Anchor Box*

Define Ground Truth Box (target) with
 (x_t, y_t) coordinates of the center of object
 (w_t, h_t) Width and height of Object

Define Anchor Box with
 (x_a, y_a) coordinates of the center of box
 (w_a, h_a) Width and height of box

[2] Regressor Vector of Anchor Box

22



To adapt **Anchor Box** size
to fit on **Object Truth Box**
(**Target Box**)

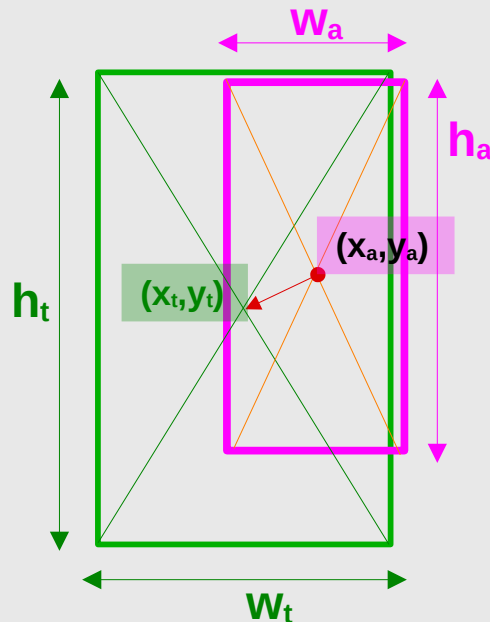
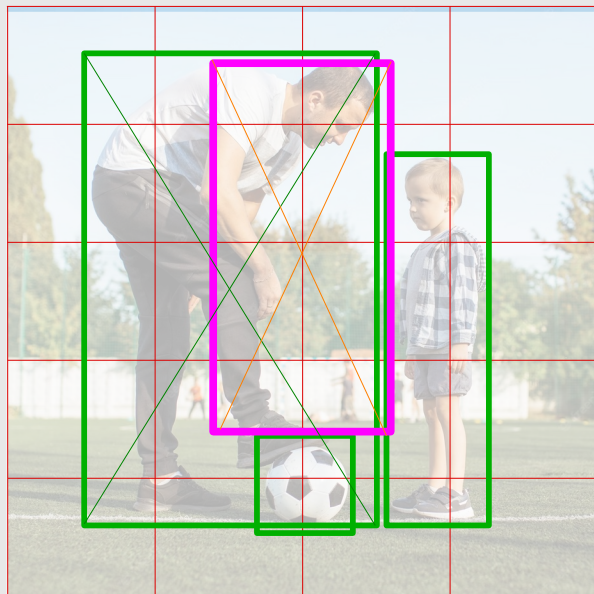
$(x_a, y_a) \rightarrow (x_t, y_t)$
displacement is $(x_t - x_a)$ and $(y_t - y_a)$

$(w_a, h_a) \rightarrow (w_t, h_t)$
Scale difference is w_t/w_a and h_t/h_a

Define Ground Truth Box (target) with
 (x_t, y_t) coordinates of the center of object
 (w_t, h_t) Width and height of Object

Define Anchor Box with
 (x_a, y_a) coordinates of the center of box
 (w_a, h_a) Width and height of box

[2] Regressor Vector of Anchor Box



To adapt **Anchor Box** size
to fit on **Object Truth Box**
(**Target Box**)

$(x_a, y_a) \rightarrow (x_t, y_t)$
displacement is $(x_t - x_a)$ and $(y_t - y_a)$

$(w_a, h_a) \rightarrow (w_t, h_t)$
Scale difference is w_t/w_a and h_t/h_a

Normalize values in order to be
object size independent

Use $d_x = (x_t - x_a)/x_a$

$d_y = (y_t - y_a)/y_a$

$d_w = w_t/w_a$

$d_h = h_t/h_a$



If **Anchor Box** fit 100%
with **Object box** then

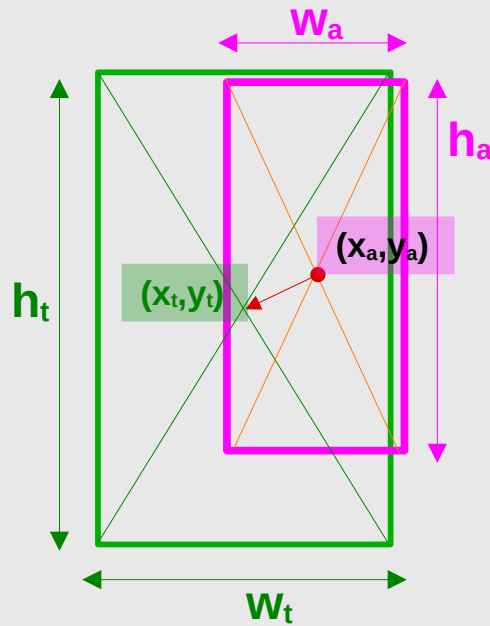
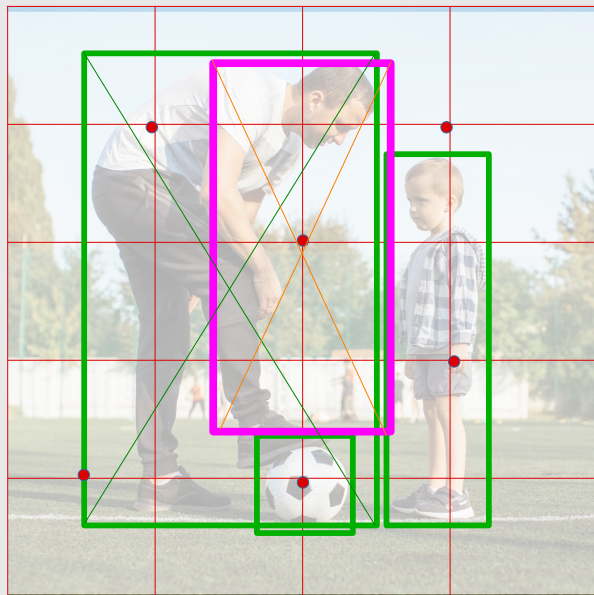
$d_x = 0$

$d_y = 0$

$d_w = 1$ -----??

$d_h = 1$ -----??

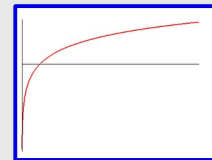
[2] Regressor Vector of Anchor Box



To adapt **Anchor Box** size
to fit on **Object Truth Box**
(**Target Box**)

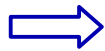
$(x_a, y_a) \rightarrow (x_t, y_t)$
displacement is $(x_t - x_a)$ and $(y_t - y_a)$

$(w_a, h_a) \rightarrow (w_t, h_t)$
Scale difference is w_t/w_a and h_t/h_a



Normalize values in order to be
object size independent

Use $d_x = (x_t - x_a)/x_a$
 $d_y = (y_t - y_a)/y_a$
 $d_w = w_t/w_a$
 $d_h = h_t/h_a$



If **Anchor Box** fit 100%
with **Object** box then

$d_x = 0$
 $d_y = 0$
 $d_w = 1$ ----- ??
 $d_h = 1$ ----- ??



Use **Log function**

$d_x = (x_t - x_a)/x_a$
 $d_y = (y_t - y_a)/y_a$
 $d_w = \log(w_t/w_a)$
 $d_h = \log(h_t/h_a)$

If **Anchor Box** fit 100%
with **Object** box then

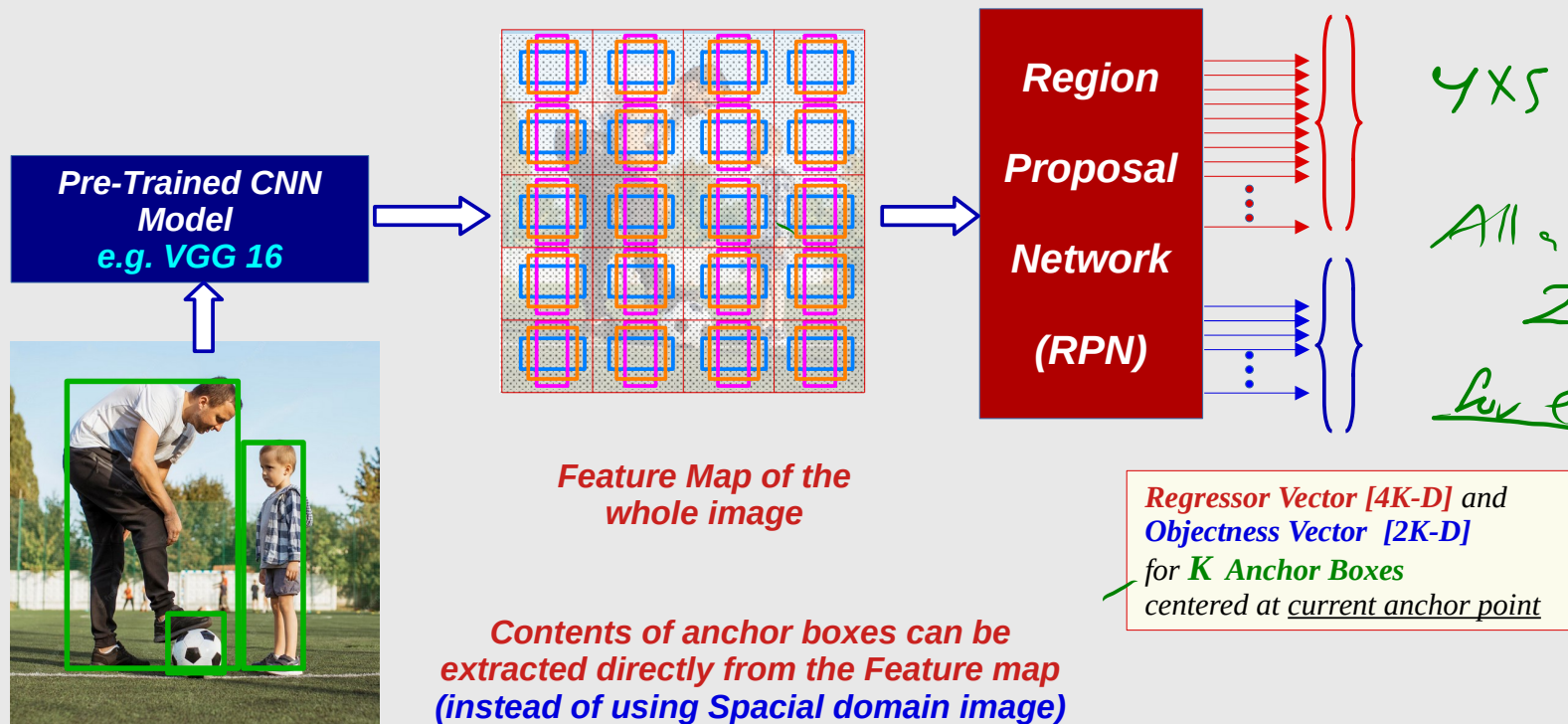
$d_x = 0$
 $d_y = 0$
 $d_w = 0$
 $d_h = 0$

All Values are **zeros**

[3] RPN Training Data Preparation

Training of RPN (The Complete Picture)

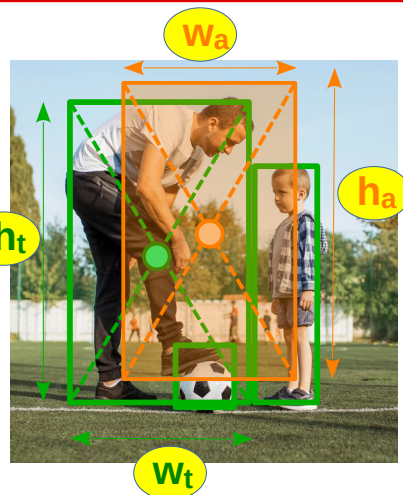
26



1st Anchor box =
9 / anchor point
 $4 \times 5 = 20$ anchor point
All anchor boxes =
 $20 \times 9 = 180$
For each $\begin{bmatrix} = \\ - \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix}$ objects
 $\frac{dx}{dy} \begin{bmatrix} = \\ - \end{bmatrix}$

Training of RPN (The Complete Picture)

27



There is an **object**
(accepted IOU)

$$d_x = (x_t - x_a) / x_a$$

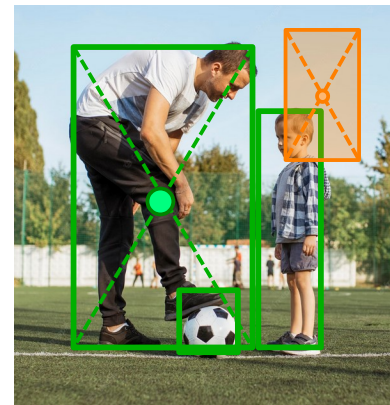
$$d_y = (y_t - y_a) / y_a$$

$$d_w = \log(w_t / w_a)$$

$$d_h = \log(h_t / h_a)$$

$$\begin{Bmatrix} 1 \\ 0 \end{Bmatrix}$$

$$\begin{Bmatrix} d_x \\ d_y \\ d_w \\ d_h \end{Bmatrix}$$



There is a **MIX**
(Medium IOU)

$$d_x = \text{Don't Care}$$

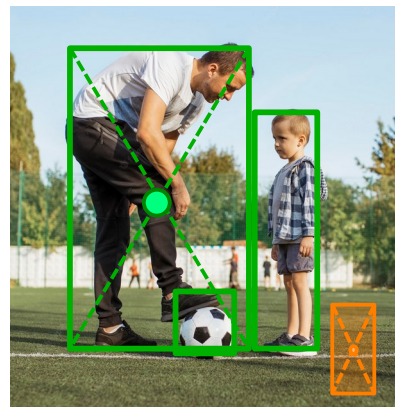
$$d_y = \text{Don't Care}$$

$$d_w = \text{Don't Care}$$

$$d_h = \text{Don't Care}$$

$$\begin{Bmatrix} 0 \\ 0 \end{Bmatrix}$$

$$\begin{Bmatrix} X \\ X \\ X \\ X \end{Bmatrix}$$



There is a **Background**
(Small IOU)

$$d_x = \text{Don't Care}$$

$$d_y = \text{Don't Care}$$

$$d_w = \text{Don't Care}$$

$$d_h = \text{Don't Care}$$

$$\begin{Bmatrix} 0 \\ 1 \end{Bmatrix}$$

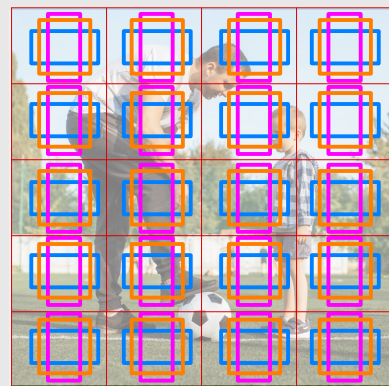
$$\begin{Bmatrix} X \\ X \\ X \\ X \end{Bmatrix}$$

Repeat for each anchor box

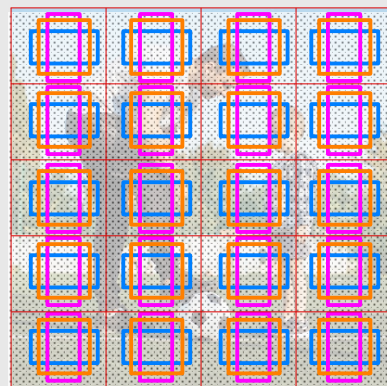
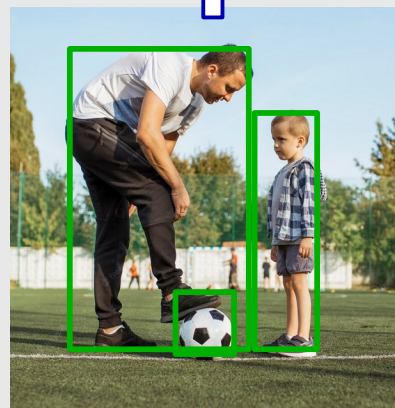
Remember:

W_a , H_a , X_a , and Y_a are known and differ from one anchor to another

Training of RPN (The Complete Picture)



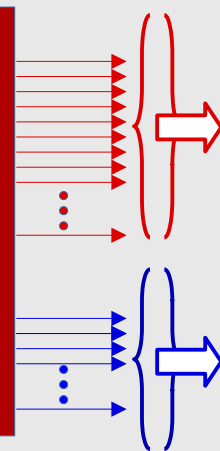
Pre-Trained CNN Model
e.g. VGG 16



Feature Map of the whole image

Contents of anchor boxes can be extracted directly from the Feature map (instead of using Spatial domain image)

Region Proposal Network (RPN)



Loss Function

Labels
For ALL anchor boxes "K"
at EACH anchor point:

Displacement vector w.r.t
overlapped Ground
Truth Box
(4 values * K anchor
boxes)
For K=9, length=36
[Regressor Vector]

Does it enclose Object,
Background, Mix ?
(2 values * K anchor
boxes)
For K=9, length=18
[Objectness Vector]

Regressor Vector [4K-D] and
Objectness Vector [2K-D]
for **K Anchor Boxes**
centered at current anchor point

How to Prepare Training Data for RPN

- 1) Get a training dataset that contain images with Annotated objects (bounding boxes)
- 2) Use any pre-trained CNN as **Backbone** (e.g. VGG16, ResNet or ResNext) to obtain the feature map of the input image
- 3) For each anchor point in the input image, generate coordinates for all “K” anchor boxes (centered at this anchor point)
- 4) Calculate IOU between those anchor boxes and ALL objects ground truth boxes.
- 5) Mark each anchor box as enclosing Object [1 , 0], Background only [0 , 1], or Mix [0 , 0].
This “Objectness” vector length should be “K”*2

How to Prepare Training Data for RPN

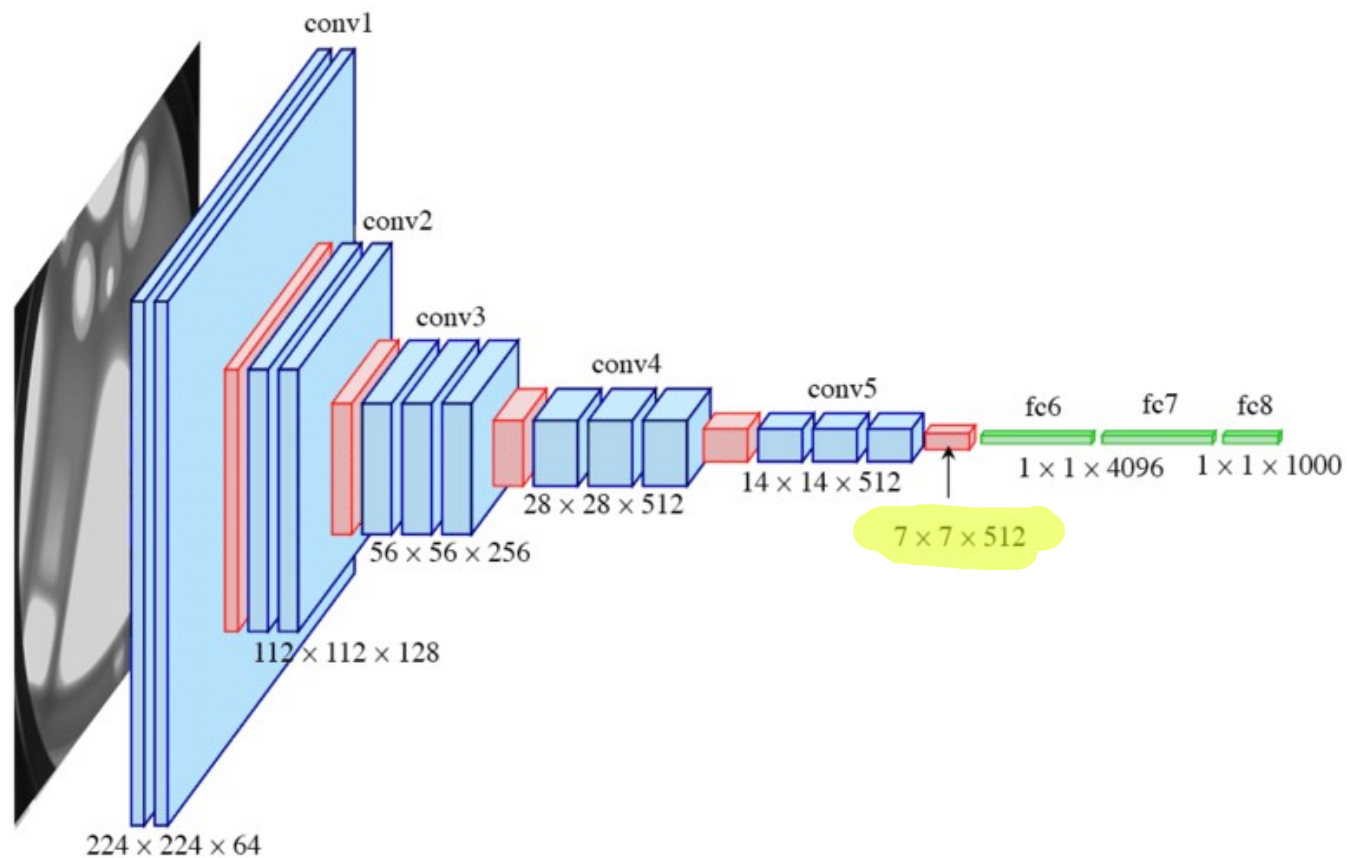
- 6) For anchor boxes including objects **ONLY**, calculate the displacement vector (4 values) between the anchor box and corresponding object box. This “Regressor” vector length should be $K*4$
- 7) For other anchor boxes (**Mix or Background**), fill the values with zeros. (will not be used). This “Regressor” vector length should be $K*4$
- 8) Provide RPN with both feature map (as Input) and “Objectness & Regressor” vectors (for each anchor point) as Label.

[4] Quick Review of VGG 16

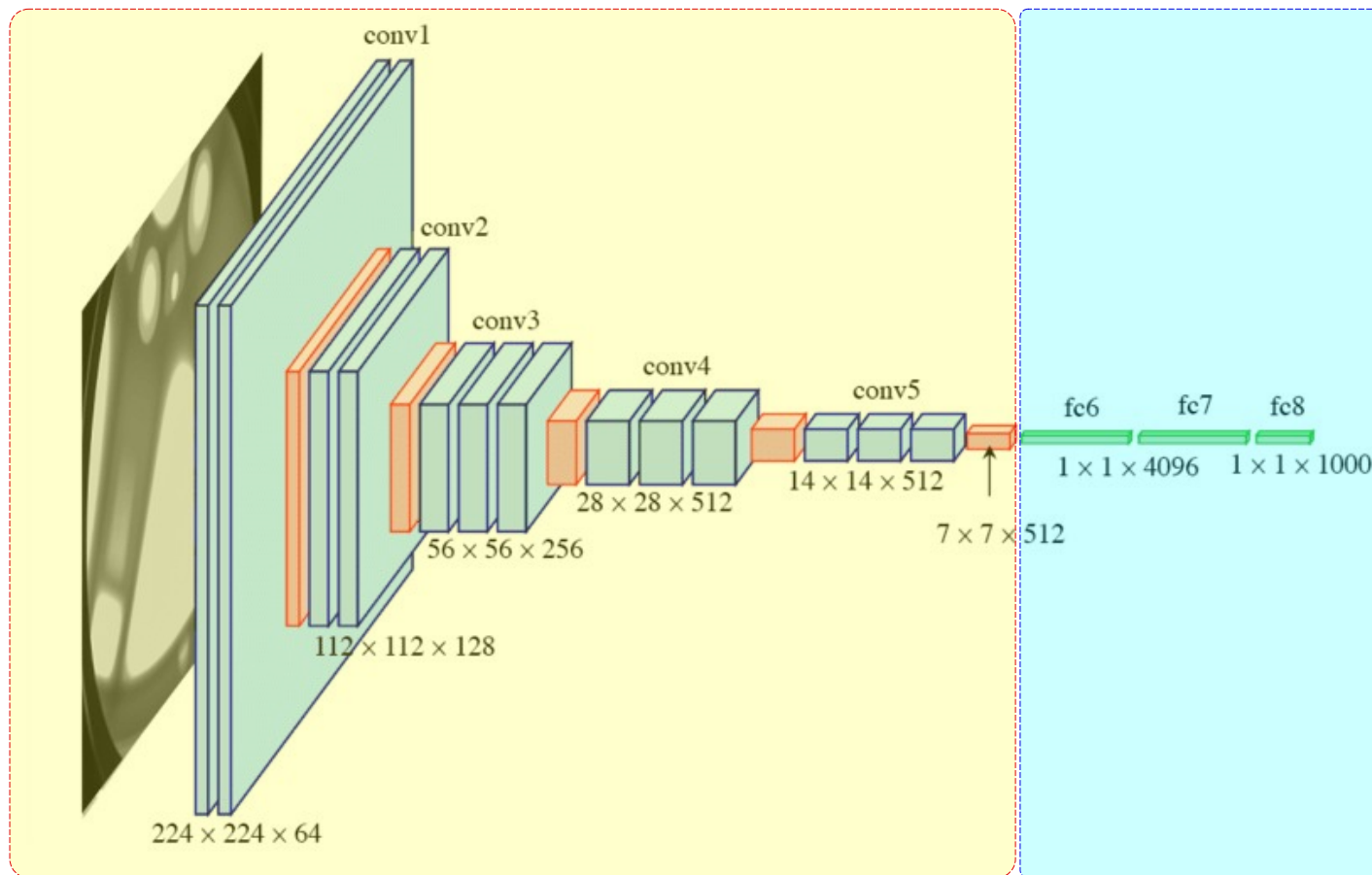
(to Generate Feature Map of Input Image)

(One may use any other CNN network as AlexNet, ResNet,..)

VGG 16 Architecture



VGG 16 Architecture



Convolution Part of VGG16

Input Image 224x224x3

Conv1: 64 filters 224x224x64
Pool 112x112x64

Conv2: 128 filters 112x112x128
Pool 56x56x128

Conv3: 256 filters 56x56x256
Pool 28x28x256

Conv4: 512 filters 28x28x512
Pool 14x14x512

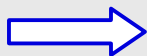
Conv5: 512 filters 14x14x512
Pool 7x7x512

- convolution+ReLU
- max pooling
- fully connected+ReLU

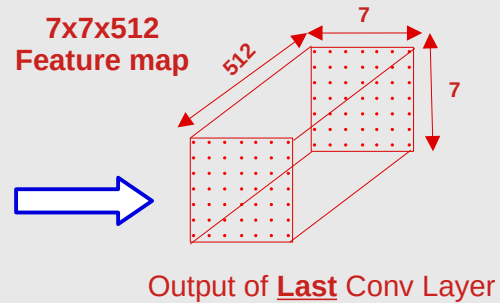
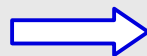
**[5] How to Apply 1x1 Convolution
*to obtain***

Regressor and Objectness Vectors

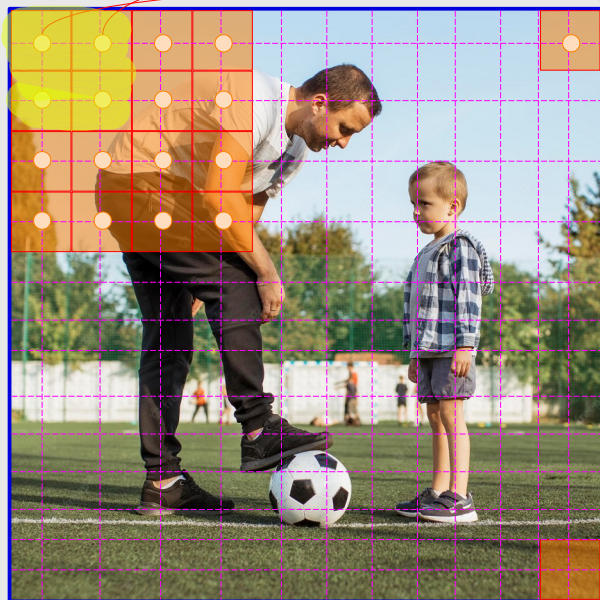
224 x 224



VGG 16
Pre-Trained CNN Model



Revision of Pooling Operation

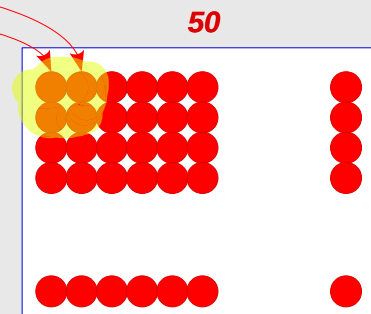


100

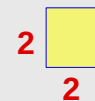
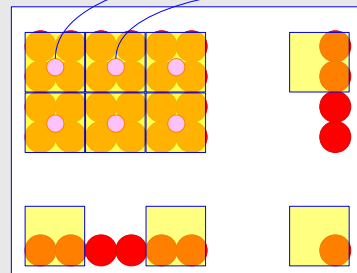
100



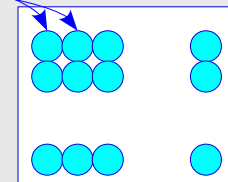
Each value covers 2x2
from the previous input



50



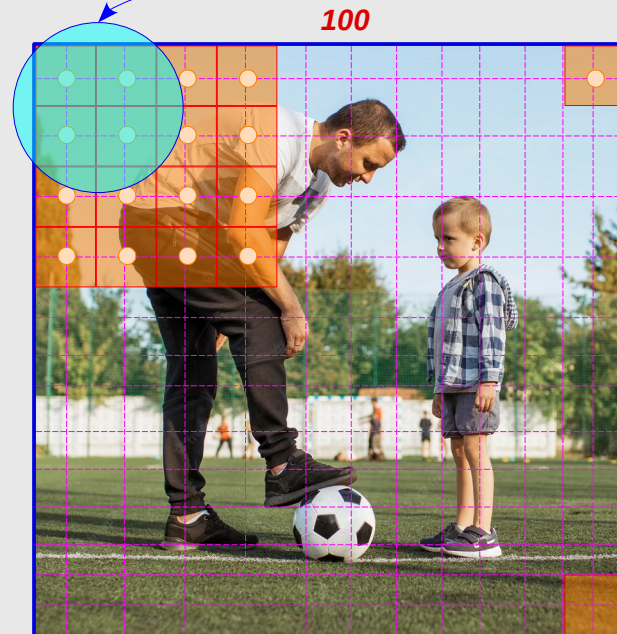
Each value covers 2x2
from the previous input



25

25

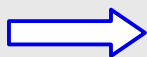
After Pooling twice
Each value covers 4x4
from the Original input



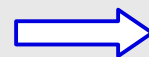
100

100

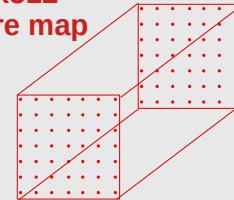
37



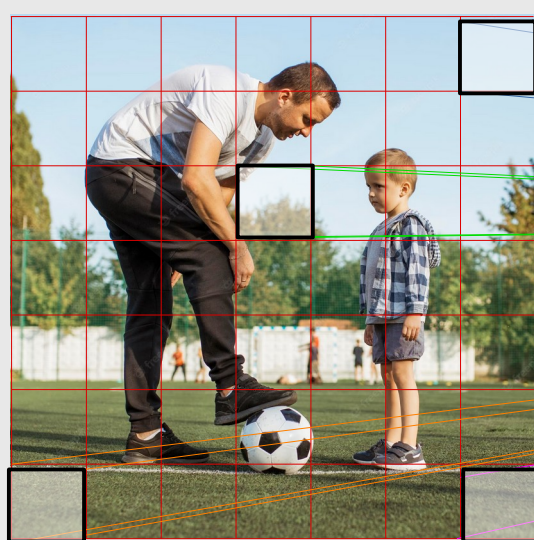
VGG 16
Pre-Trained CNN Model



7x7x512
Feature map

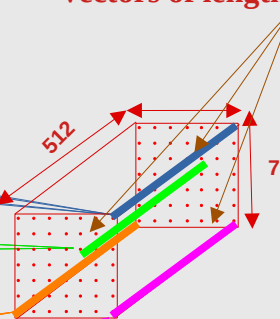


Output of Last Conv Layer



Feature map generated
from VGG 16
Last Conv Layer

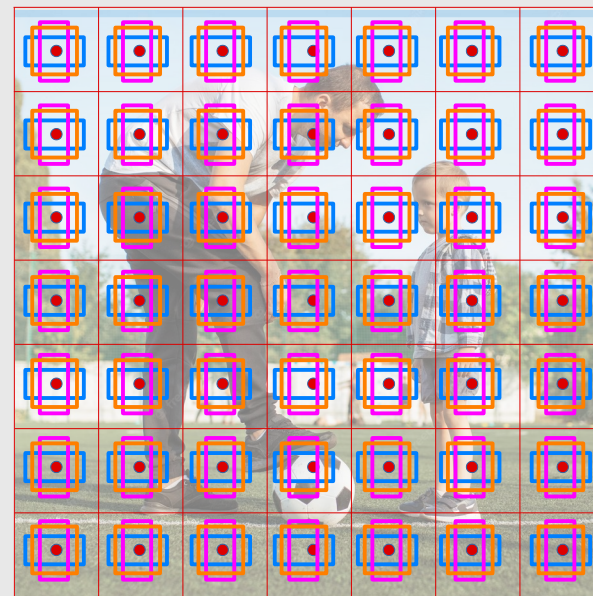
Vectors of length 512



Each Vector of size $1 \times 1 \times 512$
Is representing feature map
of block With size $224/7 \times 224/7$
From the input image

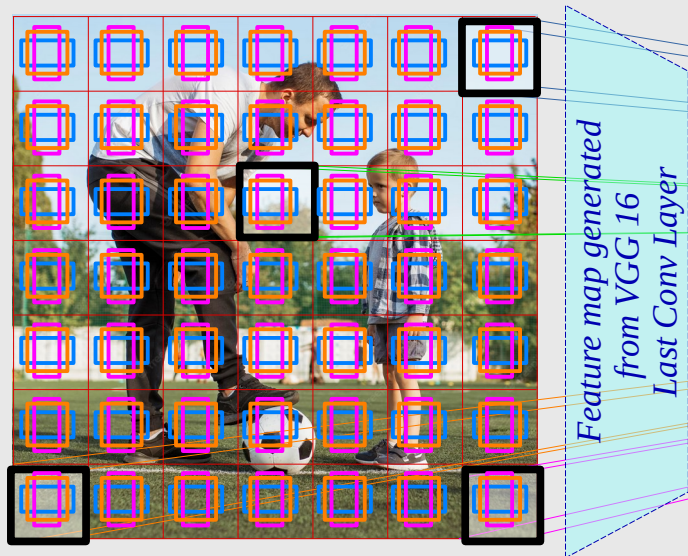
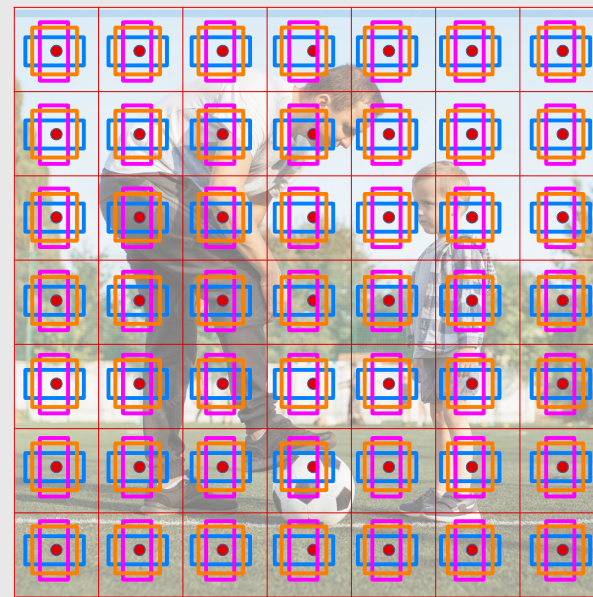
Remember:

- Input Image is divided into Cells (e.g. 7x7 cells as last Conv layer of VGG16)
- Center of each cell is an Anchor Point
- Each anchor point is associated with “K” anchor boxes (K=9 in the paper)



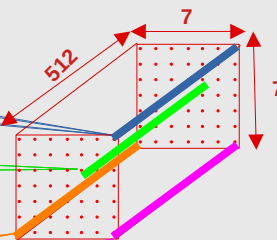
Remember:

- Input Image is divided into Cells (e.g. 7x7 cells as last Conv layer of VGG16)
- Center of each cell is an Anchor Point
- Each anchor point is associated with “K” anchor boxes (K=9 in the paper)



Feature map generated
from VGG 16
Last Conv Layer

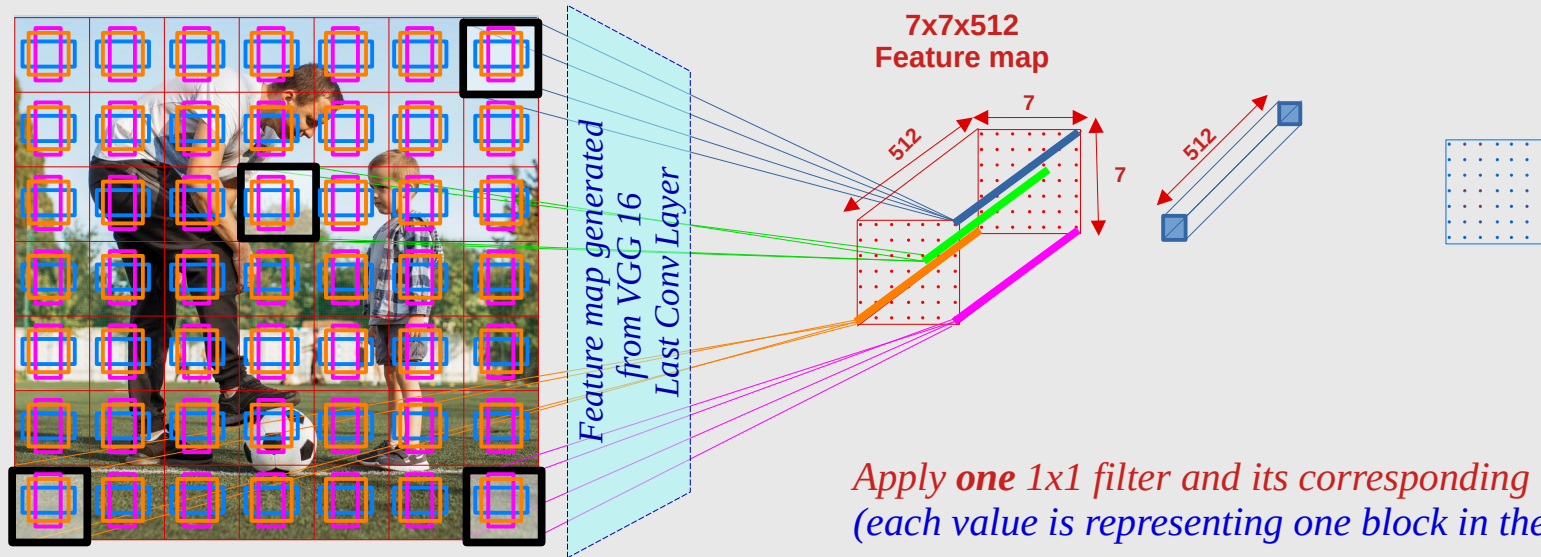
7x7x512
Feature map



Each Vector of size 1x1x512
Is representing feature map
Of block covered by “K” Anchor Boxes
(Corresponding to one Anchor Point)

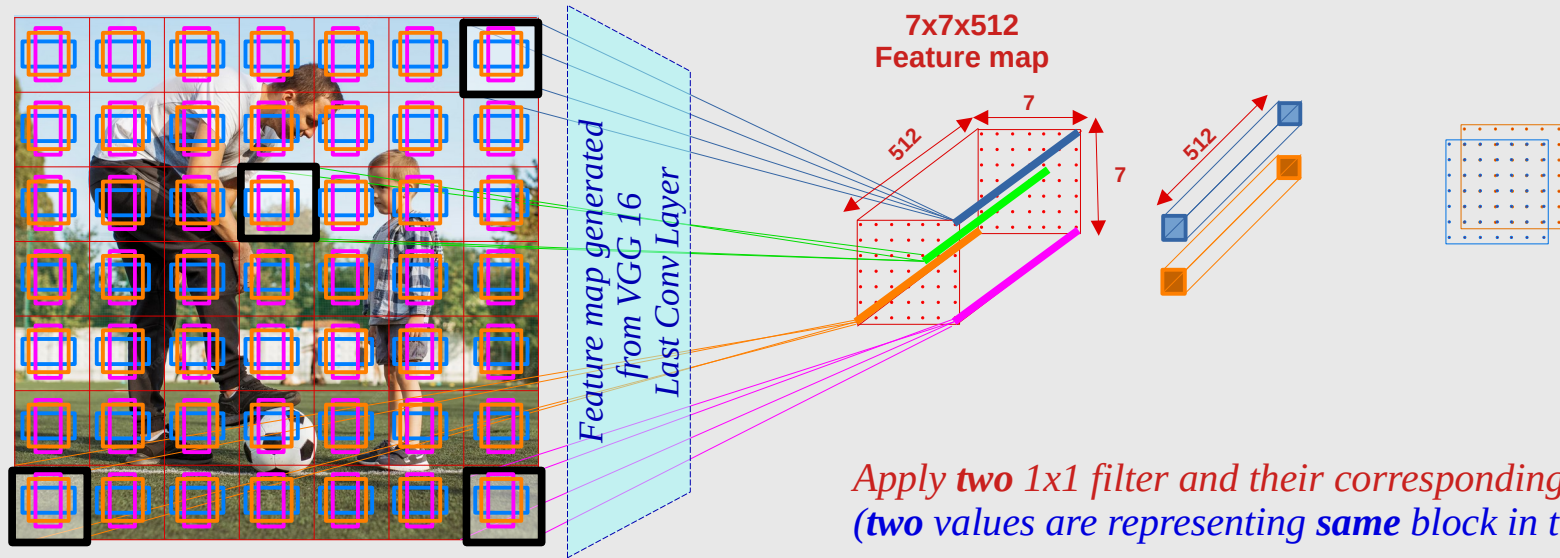
Apply 1x1 Convolution filter(s)

(on the feature map of last VGG Conv layer)



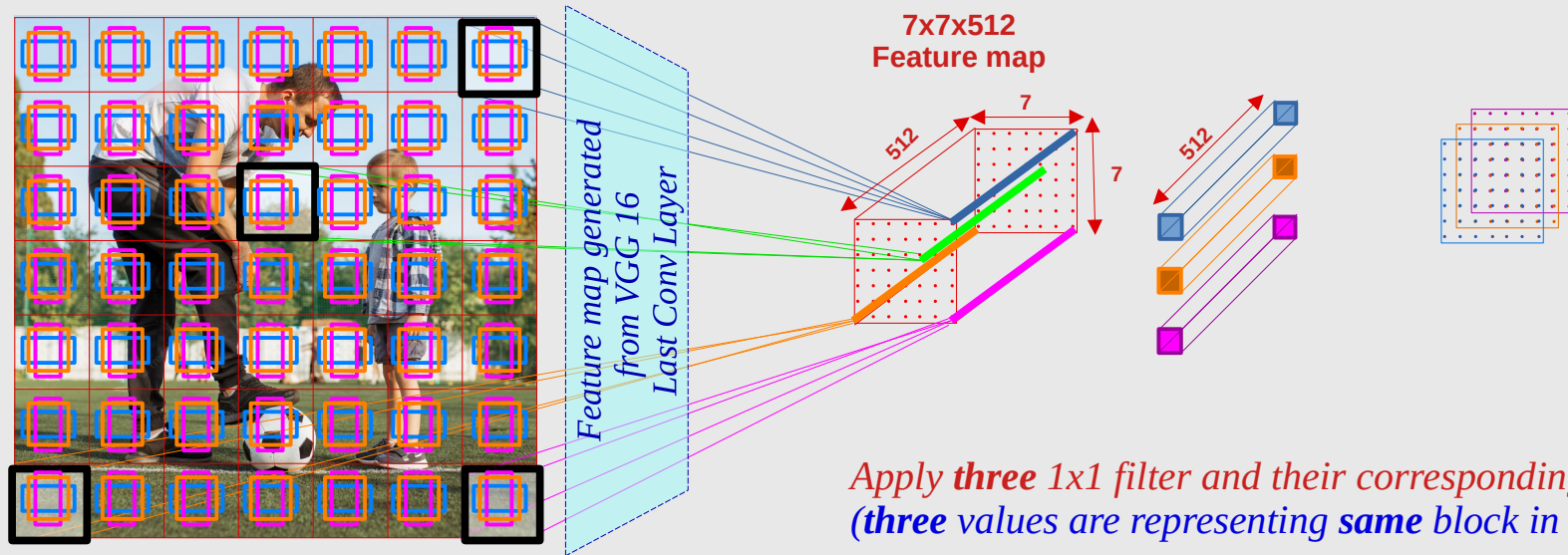
Apply 1x1 Convolution filter(s)

(on the feature map of last VGG Conv layer)



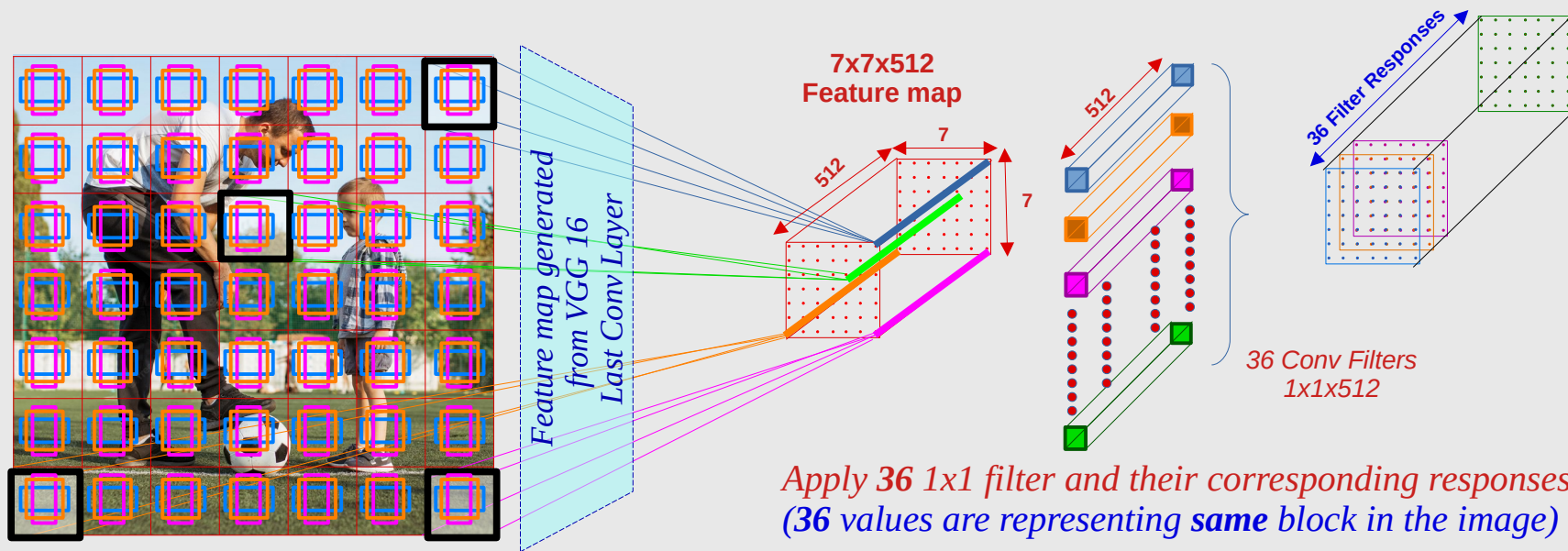
Apply 1x1 Convolution filter(s)

(on the feature map of last VGG Conv layer)

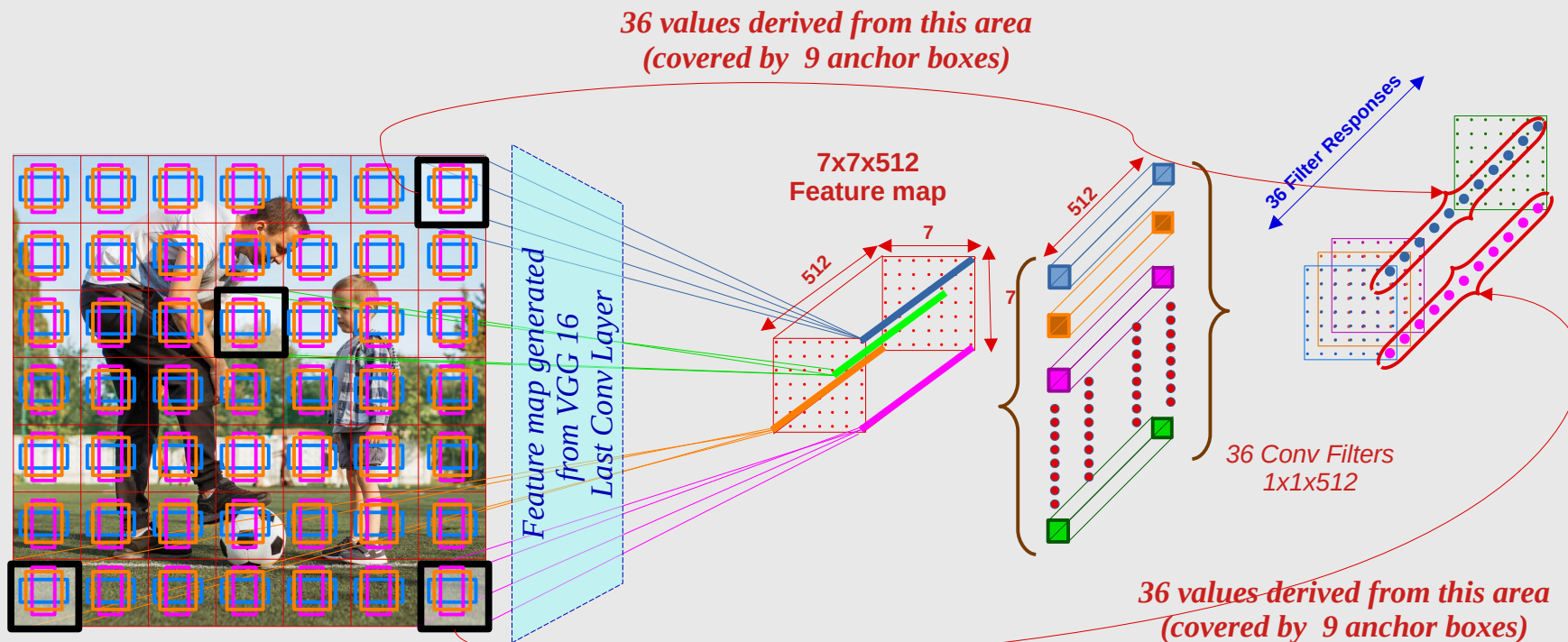


Apply 1x1 Convolution filter(s)

(on the feature map of last VGG Conv layer)



Apply 1x1 Convolution filter(s) (on the feature map of last VGG Conv layer)



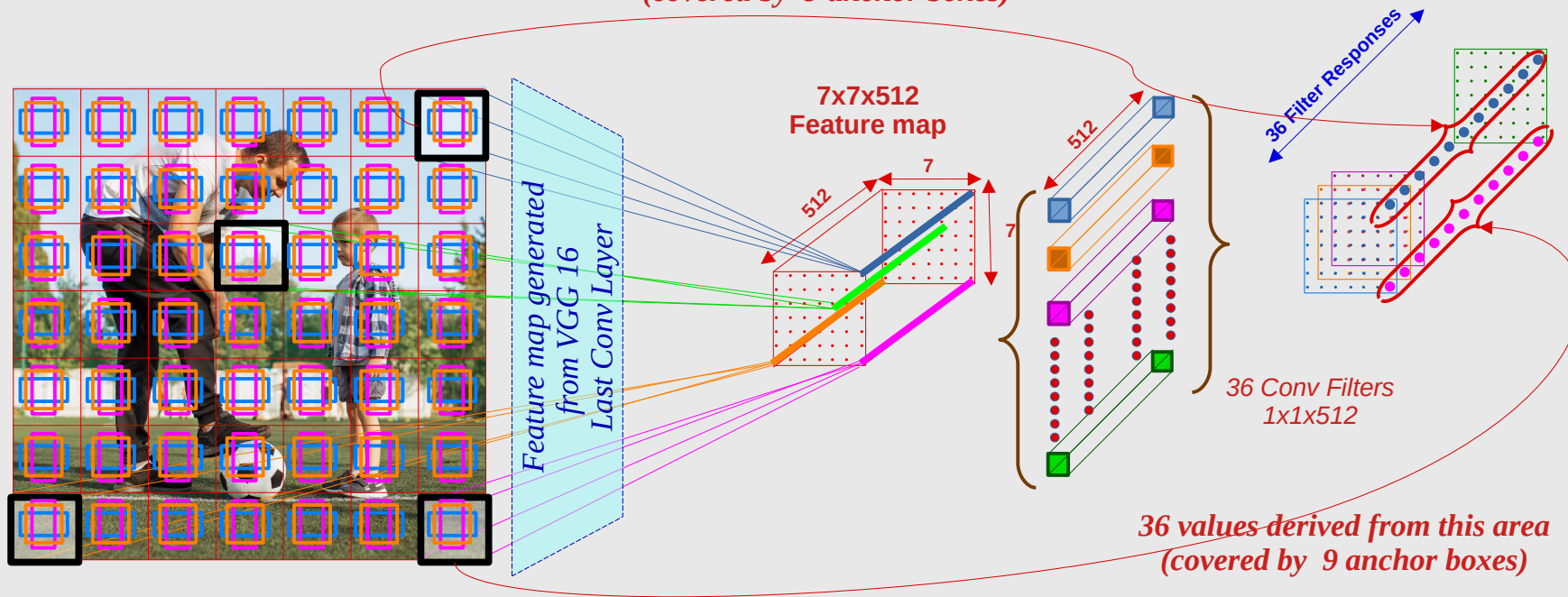
Apply 1x1 Convolution filter(s) (on the feature map of last VGG Conv layer)

Generated **49** Vectors
(there are **7 x 7** Anchor points)



Each vector of length **36**
(=9 boxes * 4 parameters)

*36 values derived from this area
(covered by 9 anchor boxes)*



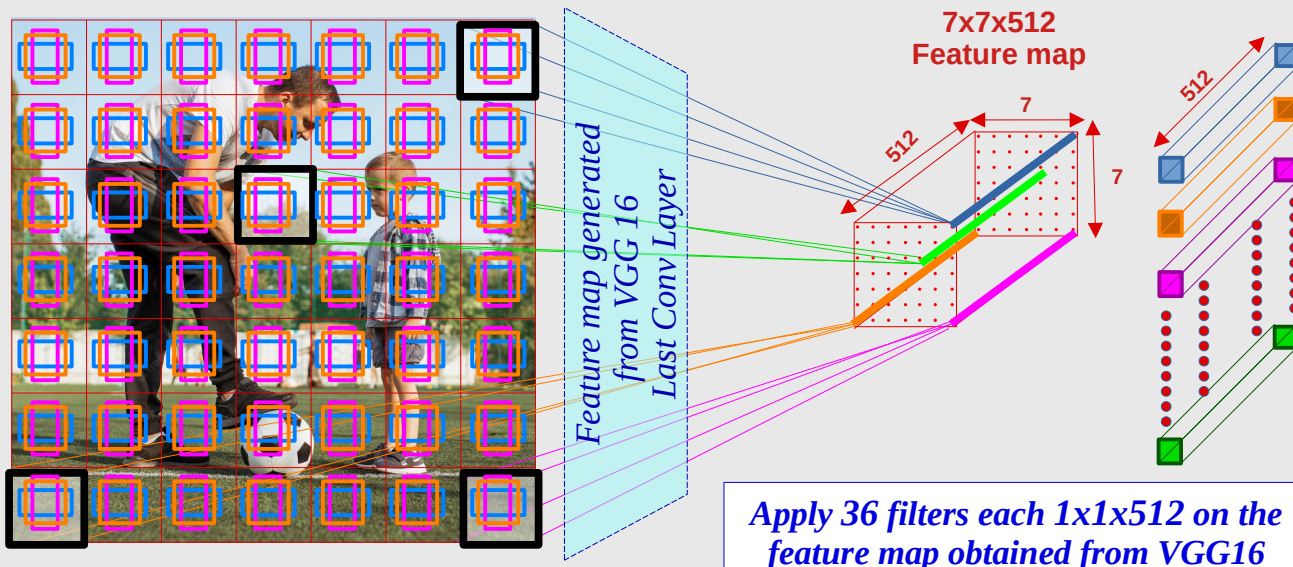
~~36 values derived from this area
(covered by 9 anchor boxes)~~

Apply 1x1 Convolution filter(s) (on the feature map of last VGG Conv layer)

Regressor Vectors

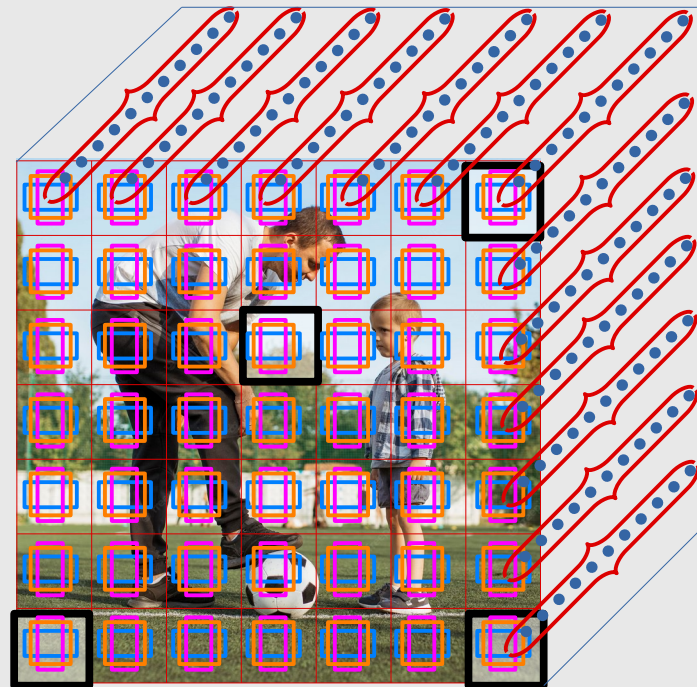
4 values per anchor box

36 values for 9 anchor boxes within same center



Apply 36 filters each 1x1x512 on the feature map obtained from VGG16

7x7 vectors each of 36 values
Each vector is derived from one area
(covered by 9 anchor boxes)

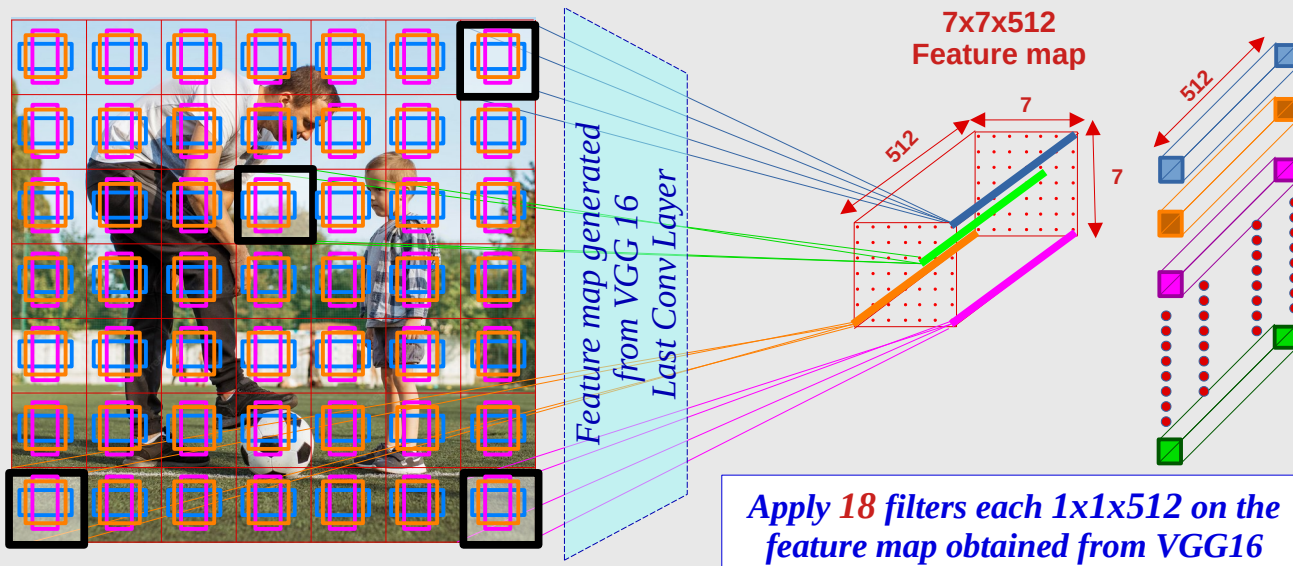


Apply 1x1 Convolution filter(s) (on the feature map of last VGG Conv layer)

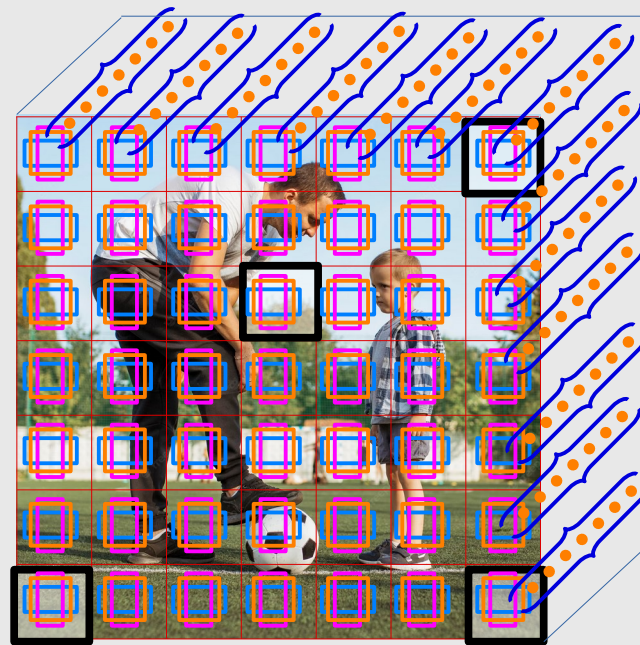
Objectness Vectors

2 values per anchor box

18 values for 9 anchor boxes within same center



7x7 vectors each of 18 values
Each vector is derived from one area
(covered by 9 anchor boxes)



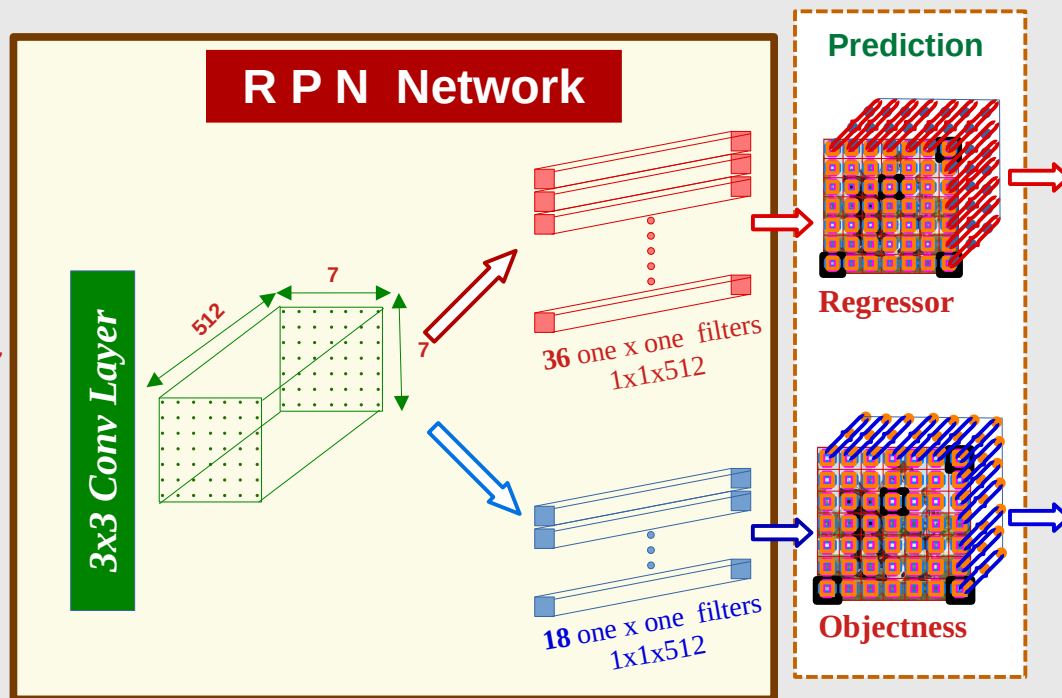
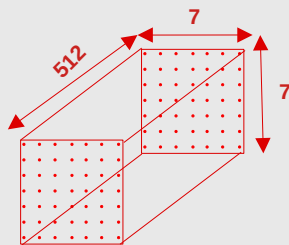
[6] RPN Architecture

RPN Architecture

(The Full Picture – Based on VGG 16 and $K=9$ anchor boxes)



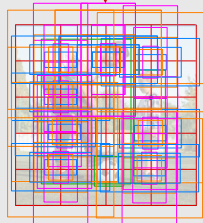
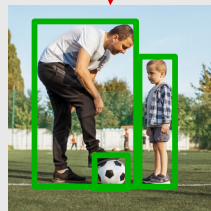
VGG 16
Output of Last Conv Layer



RPN Architecture

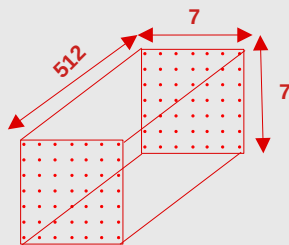
(The Full Picture – Based on VGG 16 and K=9 anchor boxes)

IOU



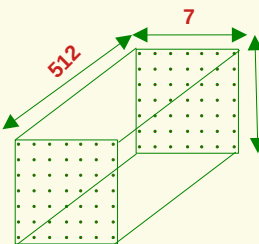
Calculate **Regressor** and **Objectness** Vectors
(one pair for each “K” anchor boxes)

VGG 16
Output of Last Conv Layer



R P N Network

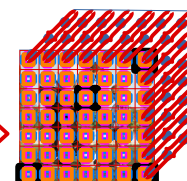
3x3 Conv Layer



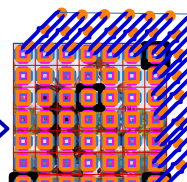
36 one x one filters
1x1x512

18 one x one filters
1x1x512

Prediction

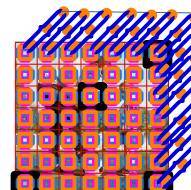
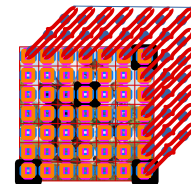


Regressor



Objectness

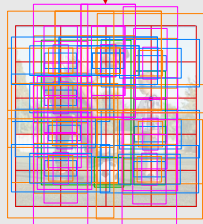
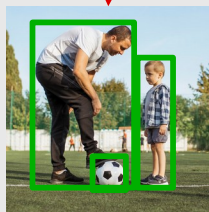
Target



RPN Architecture

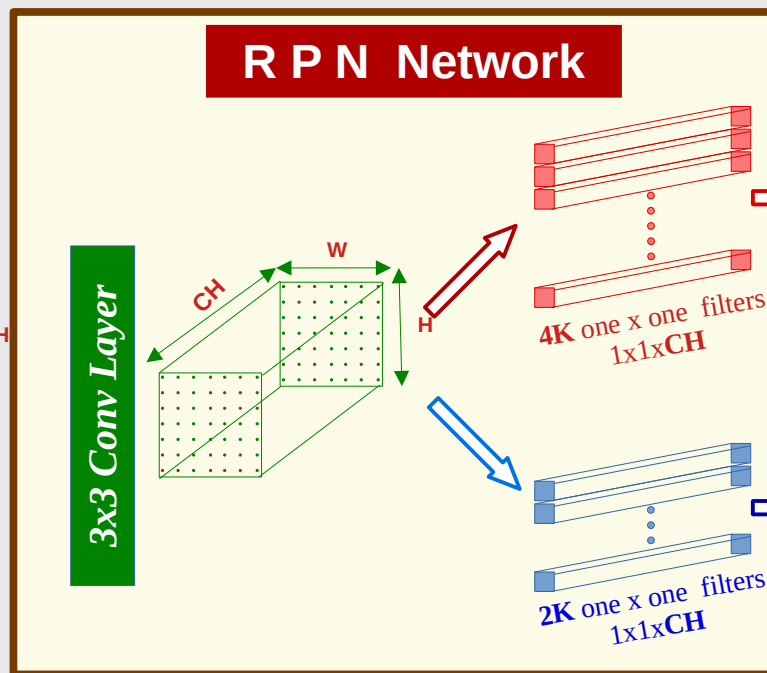
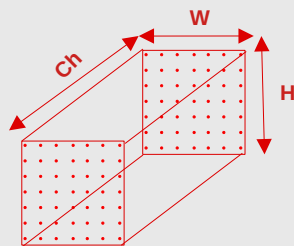
IOU

(The Full Picture – Generic Pre-trained CNN and K anchor boxes)

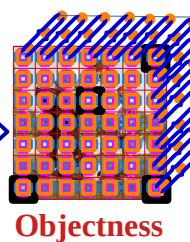
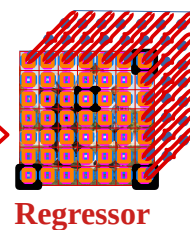


Calculate **Regressor** and **Objectness** Vectors
(one pair for each "**K**" anchor boxes)

A Pre-Trained CNN
Model
(VGG, Alex, ResNet,...)

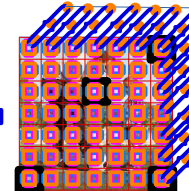
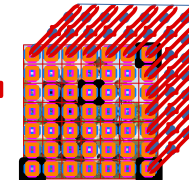


Prediction



Loss Calculations

Target

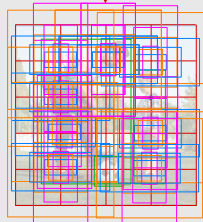
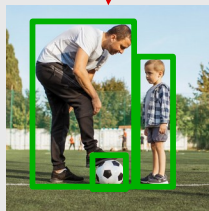


[7] RPN Loss Function

RPN Architecture

IOU

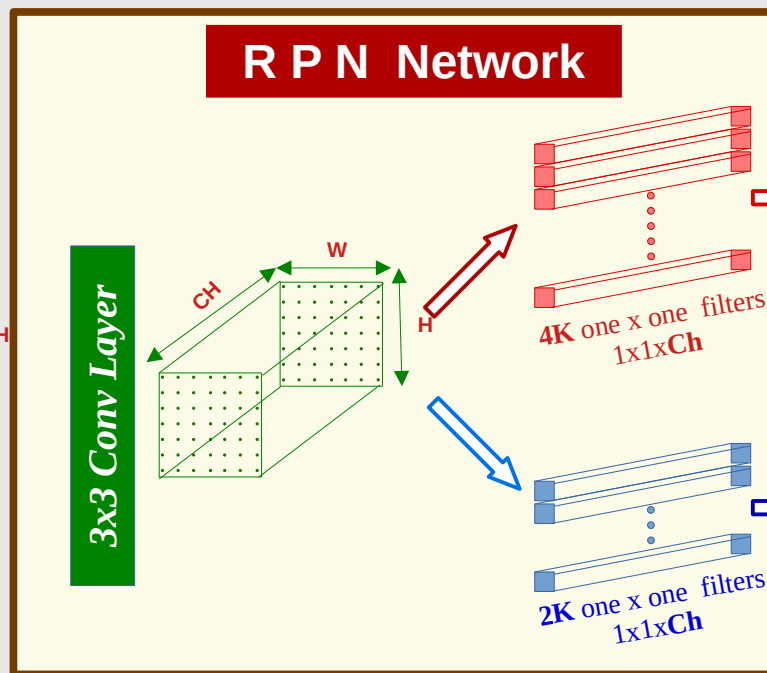
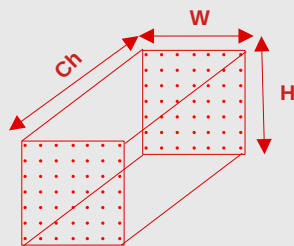
(The Full Picture – Generic Pre-trained CNN and K anchor boxes)



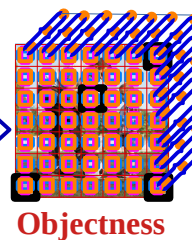
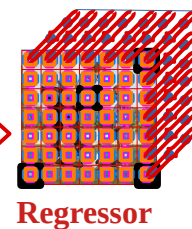
Calculate **Regressor** and **Objectness** Vectors
(one pair for each “K” anchor boxes)

Real Values
(Offsets)

A Pre-Trained CNN
Model
(VGG, Alex, ResNet,...)

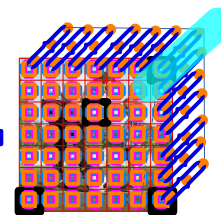
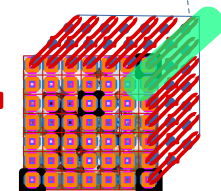


Prediction



Loss Calculations

Target

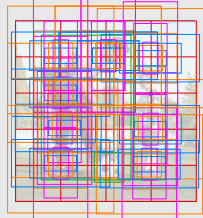
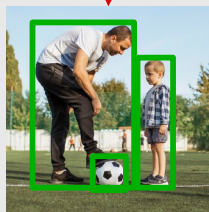


Binary
Values {0,1}

RPN Architecture

IOU

(The Full Picture – Generic Pre-trained CNN and K anchor boxes)

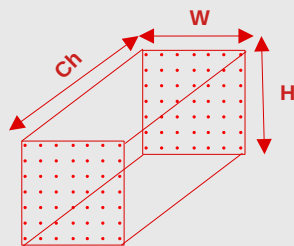


Calculate **Regressor** and **Objectness** Vectors
(one pair for each “K” anchor boxes)

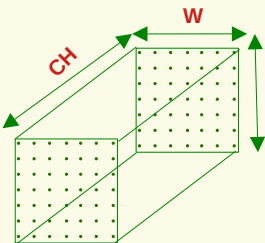
Squared Distance
Loss (L_2)

Real Values
(Offsets)

A Pre-Trained CNN
Model
(VGG, Alex, ResNet,...)



3x3 Conv Layer

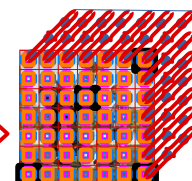


4K one x one filters
1x1xCh

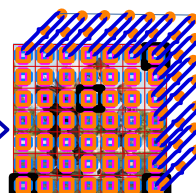
2K one x one filters
1x1xCh

R P N Network

Prediction



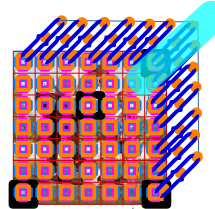
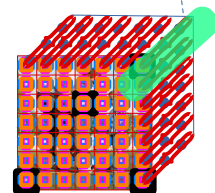
Regressor



Objectness

Loss Calculations

Target



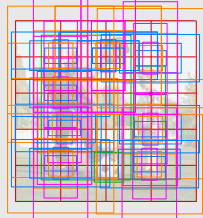
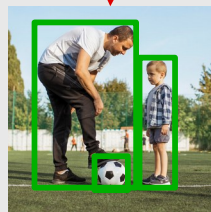
Cross Entropy Loss

Binary
Values {0,1}

RPN Architecture

IOU

(The Full Picture – Generic Pre-trained CNN and K anchor boxes)

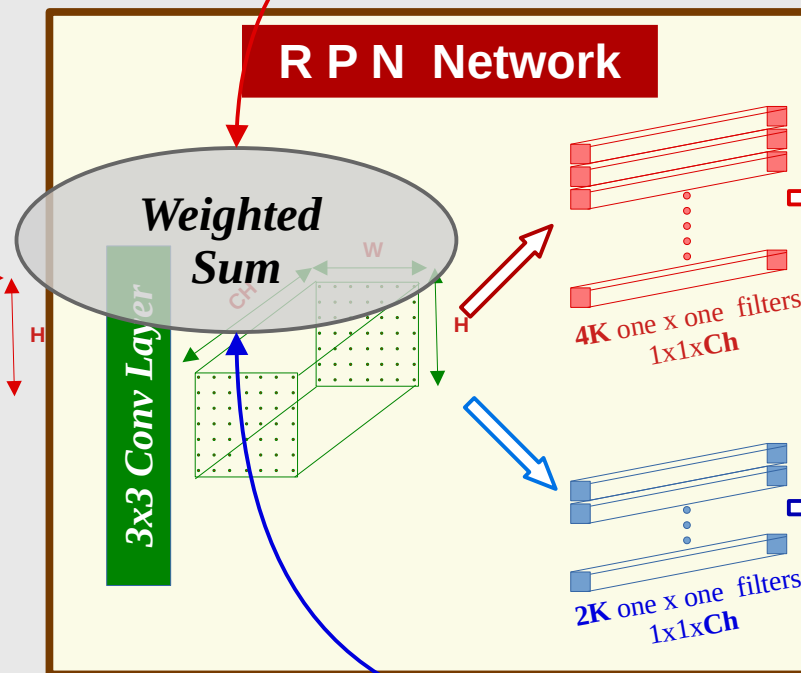
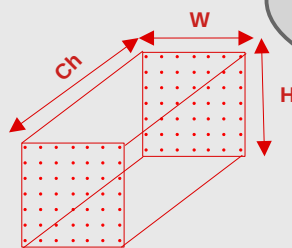


Calculate **Regressor** and **Objectness** Vectors
(one pair for each “K” anchor boxes)

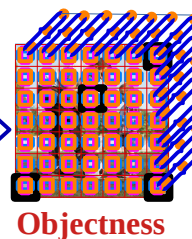
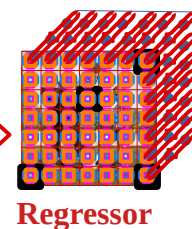
Squared Distance
Loss (L_2)

Real Values
(Offsets)

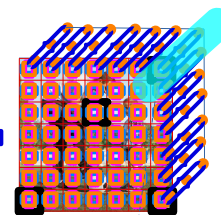
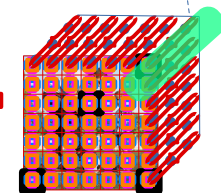
A Pre-Trained CNN
Model
(VGG, Alex, ResNet,...)



Prediction



Target



Loss Calculations

Cross Entropy Loss

Binary
Values {0,1}

[1] Object Classification Loss (Object / Background)

57

$$\text{Object classification Loss} = \frac{1}{N_{cls}} \sum_{i=1}^{N_{cls}} L_{cls}(p_i^L, p_i^P)$$

Where

N_{cls} Number of Samples used to calculate classification loss

L_{cls} Classification Loss Function

p_i^L Probabilities Vector of *target* sample number (i)

{ 1 = Positive (Object) 0 = Negative (Background) }

p_i^P Predicted Probabilities Vector that *anchor* sample number (i)

{ Prob that it contains an object and it contains Background }

L_{cls} is log Loss (Cross Entropy Loss) since the target vector is *one-hot* (Probability)

$$L_{cls}(p^L, p^P) = -p^L \log p^P$$

[2] Box Regressor Loss

$$\text{Box Regressor Loss} = \frac{1}{N_{\text{reg}}} \sum_{i=1}^N p_i^t L_{\text{reg}}(d_i^L, d_i^P)$$

Where

N_{reg} Number of **Positive** Samples used to calculate regression loss

L_{reg} Regressor Loss Function

p_i^t Probability of **target** sample number (i) is Object

Summation over boxes containing **objects ONLY**

d_i^P offsets vector of **anchor** sample number (i) [Predicted]

d_i^L offsets vector of **target** sample number (i) [Label]

$$L_{\text{reg}}(d^L, d^P) = L_1(d_x^P - d_x^L) + L_1(d_y^P - d_y^L) + L_1(d_w^P - d_w^L) + L_1(d_h^P - d_h^L)$$

L₁ ???

Remember:
(for label vector)

$$d_x = (x_t - x_a) / x_a$$

$$d_y = (y_t - y_a) / y_a$$

$$d_w = \log(w_t / w_a)$$

$$d_h = \log(h_t / h_a)$$

**To retrieve Target
Parameters from
predicted parameters**

$$X_t = d_x x_a + x_a$$

$$Y_t = d_y * y_a + y_a$$

$$w_t = e^{(d_w)} * w_a$$

$$h_t = e^{(d_h)} * h_a$$

Smoothing Function (L_1)

59

Where L_1 is a **smoothing** L_1 function:

$$L_1 = \begin{cases} 0.5 x^2 & \text{if } |x| < 1 \\ |x| - 0.5 & \text{otherwise} \end{cases}$$

Example:

$x = 0.5$ $x^2_{L_1} = 0.25$

$x = 0.1$ $x^2_{L_1} = 0.01$

$x = 0.01$ $x^2_{L_1} = 0.0001$

 $x = 1$ $x^2_{L_1} = 0.5$

$x = -1$ $x^2_{L_1} = 0.5$

For Small Values of X between $\{-1, 1\}$
Output is between $\{0, 0.5\}$ Loss is reduced

For Large Values of X $\text{Abs}(X) > 1$
output is almost the same (subtract 0.5 for function continuity)

Smoothing Function (L_1)

60

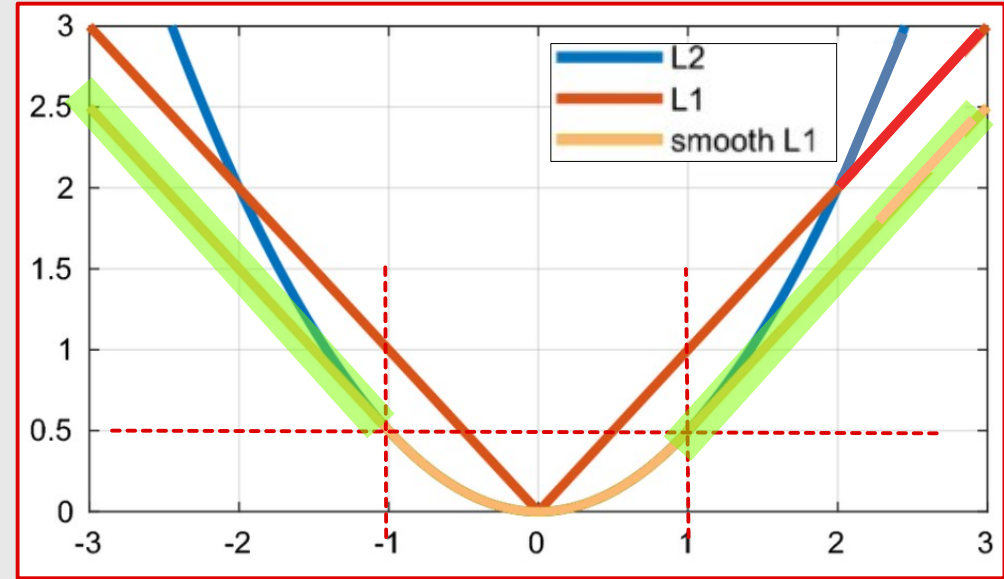
Where L_1 is a *smoothing* L_1 function:

$$L_1 = \begin{cases} 0.5x^2 & \text{if } |x| < 1 \\ |x| - 0.5 & \text{otherwise} \end{cases}$$

Where:

L_1 is Absolute distance

L_2 is Squared distance



For Small Values of X between $\{-1, 1\}$
Output is between $\{0, 0.5\}$ Loss is reduced

For Large Values of X $\text{Abs}(X) > 1$
output is almost the same (subtract 0.5 for function continuity)

Total Loss Function

$$\text{Loss Function} = \text{Object classification Loss} + \lambda * \text{Box Regressor Loss}$$

λ is a weight parameter

$$\text{Object classification Loss} = \frac{1}{N_{cls}} \sum_{i=1}^{N_{cls}} L_{cls}(p_i^L, p_i^P)$$

Where:

$$L_{cls}(p^L, p^P) = -p^L \log p^P$$

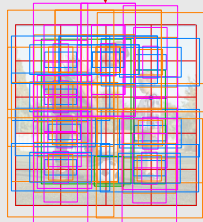
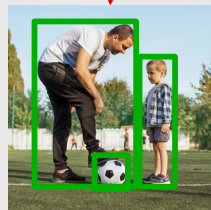
$$\text{Box Regressor Loss} = \frac{1}{N_{reg}} \sum_{i=1}^N p_i^t L_{reg}(d_i^L, d_i^P)$$

Where:

$$L_{reg}(d^L, d^P) = L_1(d_x^P - d_x^L) + L_1(d_y^P - d_y^L) + L_1(d_w^P - d_w^L) + L_1(d_h^P - d_h^L)$$

Loss Calculations on Mini-Batch

IOU

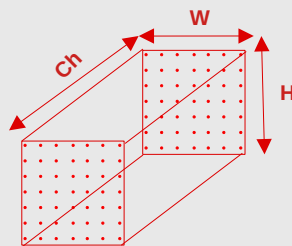


Calculate **Regressor** and **Objectness** Vectors
(one pair for each “K” anchor boxes)

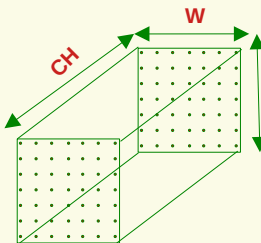
Select 128 Positive and 128 Negative Boxes Only for Loss calculations
Don't use ALL anchor boxes

ONLY 256 labels

A Pre-Trained CNN Model
(VGG, Alex, ResNet,...)



3x3 Conv Layer

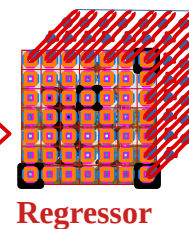


4K one x one filters
1x1xCh

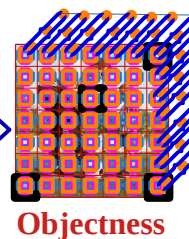
2K one x one filters
1x1xCh

RPN Network

Prediction



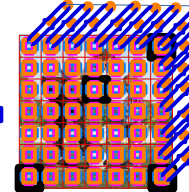
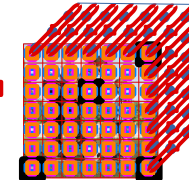
Regressor



Objectness

Loss Calculations

Target

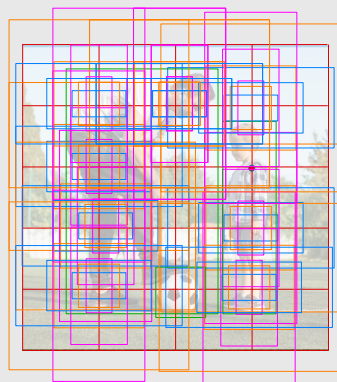


Remember: The whole image is trained **One Shot**

Final Region Proposals

using RPN and Non-Max Suppression

Obtain Region Proposals (from RPN) (for ALL Anchor Boxes)



For each Anchor Point
 (x_a, y_a) are predefined

For each Anchor Box
 (w_a, h_a) are predefined



$$X_t = d_x x_a + x_a$$

$$Y_t = d_y * y_a + y_a$$

$$w_t = e^{(d_w)} * w_a$$

$$h_t = e^{(d_h)} * h_a$$

Remember

*Model Outputs are
(for each anchor Box)*

$$d_x = (x_t - x_a) / x_a$$

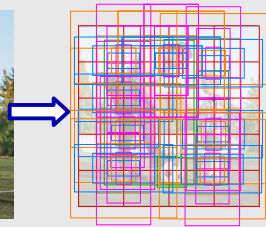
$$d_y = (y_t - y_a) / y_a$$

$$d_w = \log(w_t / w_a)$$

$$d_h = \log(h_t / h_a)$$



Obtain Region Proposals (from RPN) (for ALL Anchor Boxes)

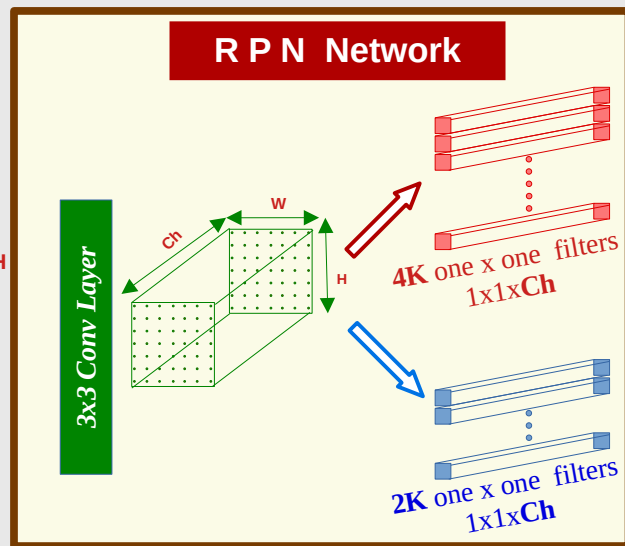
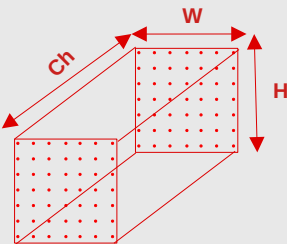


calculate parameters of anchor boxes
($\mathbf{x}_a, \mathbf{y}_a, \mathbf{w}_a, \mathbf{h}_a$) for ALL $W \times H \times K$ boxes
($7 \times 7 \times 9$) for VGG16 example

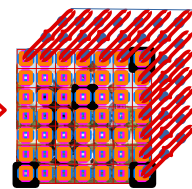
$$\begin{aligned} X_t &= d_x x_a + x_a \\ Y_t &= d_y * y_a + y_a \\ w_t &= e^{(d_w)} * w_a \\ h_t &= e^{(d_h)} * h_a \end{aligned}$$



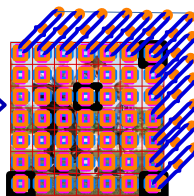
A Pre-Trained CNN
Model
(VGG, Alex, ResNet,...)



Prediction



Regressor



Objectness

Get Predicted
regressor vectors for
ALL Anchor boxes
(d_x, d_y, d_w, d_h)

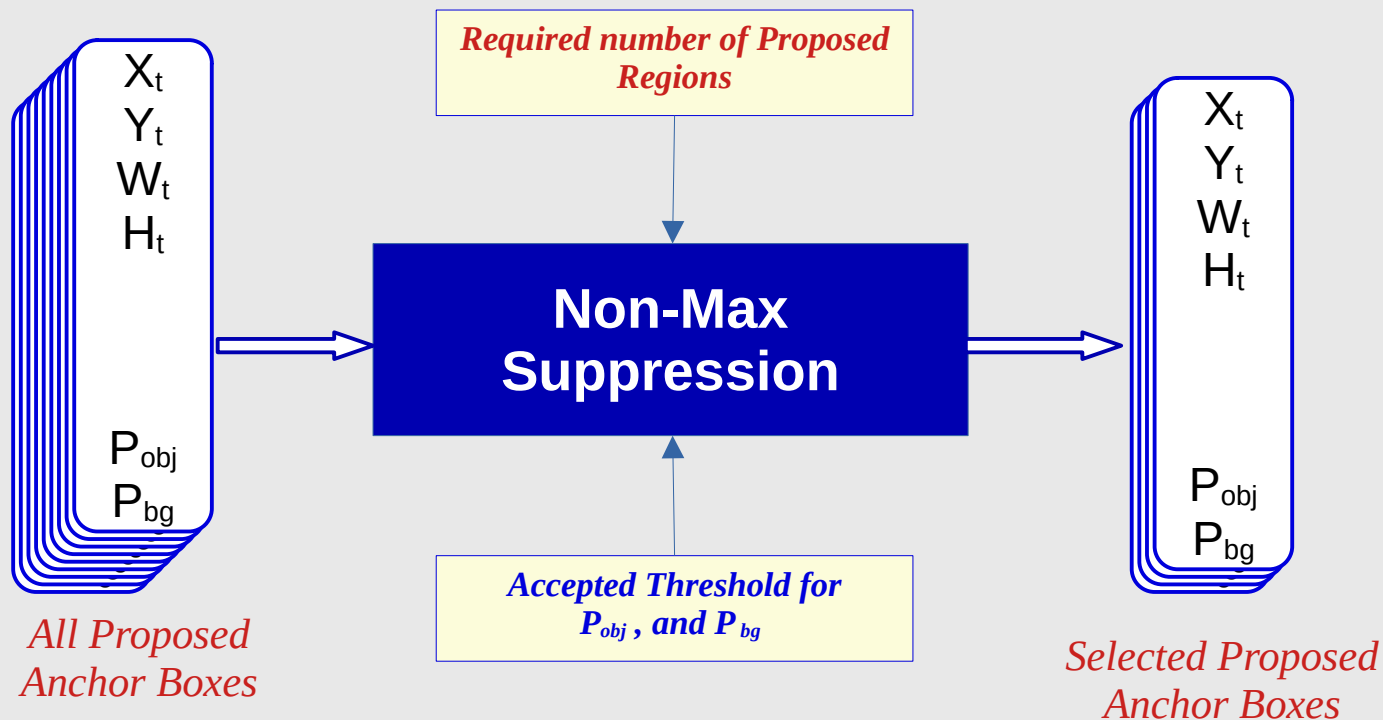
Get Predicted
Classification vectors
for ALL Anchor boxes
(p_{obj}, p_{bg})

X_t
 Y_t
 W_t
 H_t

P_{obj}
 P_{bg}

Filter RPN Region Proposals

(Remove highly overlapped Anchor Boxes)



Faster R-CNN

FAST R-CNN + (Region proposal Network)

